



UNIVERSITÀ DEGLI STUDI DI CATANIA

DIPARTIMENTO DI MATEMATICA E INFORMATICA

CORSO DI LAUREA MAGISTRALE IN INFORMATICA

Petronio Petronio Davide

Principi di Programmazione Parallela

APPUNTI LEZIONI

Prof. Vitello e Prof.ssa Sciacca

Anno Accademico 2025 - 2026

Indice

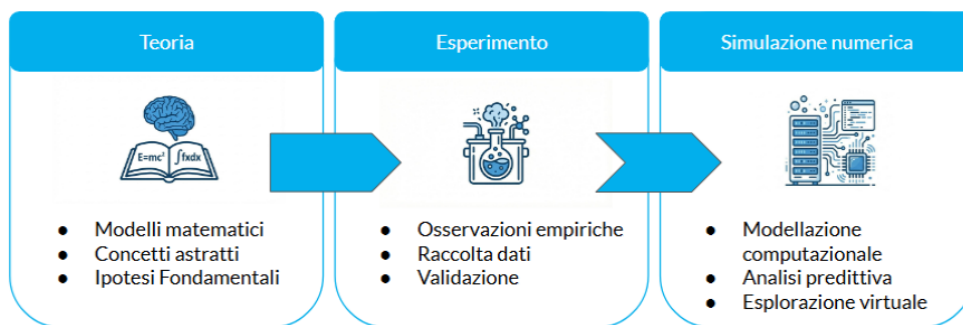
1	Architetture Moderne	7
1.1	Memory Wall	8
1.1.1	Gerarchia di Memoria	8
1.1.2	Prefetching	9
1.2	Parallelismo Interno al Processore	9
1.2.1	Pipelining	9
1.2.2	Instruction-Level Parallelism (ILP)	10
1.2.3	SIMD: Parallelismo Vettoriale	10
1.2.4	SMT: Simultaneous Multithreading	11
1.3	Sistemi Multicore e Modelli di Accesso alla Memoria	11
1.3.1	Il Modello MIMD	12
1.3.2	SMP vs NUMA	12
1.4	Sistemi a Memoria Condivisa e Distribuita	12
1.4.1	Sistemi Ibridi e HPC	13
1.4.2	Reti e Acceleratori	13
1.5	Modelli di Analisi: Il Roofline Model	14
2	Software Stack in HPC	15
2.1	Filesystem nei sistemi HPC	17
2.2	Compilatori in ambienti HPC	18
2.3	Librerie	19
2.4	Scheduler	20
2.5	Workflow per sistemi HPC	21
3	Ottimizzazione	23
3.1	Misurazione delle performance	23
3.1.1	Misure sul clock	23
3.1.2	Misure sul Tempo	24
3.1.3	Profiling	25
3.2	Ottimizzare	25
3.2.1	Memory Aliasing	28

4	Ottimizzazione Single-Core	30
4.1	Memoria cache	30
4.2	CPU	32
4.2.1	Pipeline di Esecuzione	32
4.3	Evitare le Inefficienze	34
4.3.1	Esempio che seguiremo	34
4.3.2	1. Evitare Chiamate a Funzioni Costose	34
4.3.3	2. Hoisting delle espressioni	35
4.3.4	3. Chiarire lo Scope delle Variabili	36
4.3.5	4. Suggestisci cosa è Importante	37
4.3.6	5. Non ripetere controlli o referenziamenti non necessarie	37
4.3.7	Avvertenza su Numeri in Virgola	39
4.4	Ottimizzazione basata sulla Cache	39
4.4.1	Mappatura della Cache	40
4.4.2	Modello di accesso alla Memoria	41
4.4.3	Organizzazione dei Dati	43
4.5	Branch condizionali	45
4.5.1	Branch Predictor	46
4.5.2	Tecniche per limitare i Branch	47
4.6	Ottimizzazione dei Loop	49
4.6.1	Ottimizzazioni per Cicli a Bassa Intensità: Loop Fusion	50
4.6.2	Ottimizzazioni per Cicli $O(N^2)$: Unroll & Jam	51
4.6.3	Srotolamento del Ciclo (Loop Unrolling) e Aritmetica Floating Point	51
4.6.4	Esempio Pratico $O(N^3)/O(N^2)$: Moltiplicazione Matrice-Matrice (MMM)	52
5	Parallelismo e Scalabilità	54
5.1	Modelli di parallelismo	54
5.2	Scalabilità	56
6	OpenMP	59
6.1	Cosa è OpenMP	60
6.2	Modello Fork-Join	61
6.2.1	Direttive	62
6.3	Regioni Parallele	63
6.3.1	Gestione della Regione Parallela	65

Introduzione

Con HPC intendiamo ci riferiamo a **High-Performance Computing** che rappresenta il calcolo ad alte prestazioni. Il problema nasce dal fatto che, al giorno d'oggi, il computer tradizionale non basta più a causa di:

- **Limiti del singolo computer:** al giorno d'oggi, i problemi sono troppo grandi per un singolo computer e un singolo computer è troppo lento.
- **Vincoli Sperimentali:** I problemi sono troppo costosi o impossibile da testare sperimentalmente.
- **Terzo pilastro:** La simulazione numerica è, piano piano, diventato un pilastro della scienza grazie al fatto che rende possibile effettuare calcoli e testare funzioni sperimentali a supporto del metodo scientifico.



Tramite l'HPC possiamo andare a superare i problemi del calcolo sequenziale attraverso il **parallelismo** che diventa quindi una necessità strutturale a causa di quanto detto precedentemente. Infatti, quello che accade molto spesso è che i problemi hanno un fattore di crescita **non lineare** rendendo la computazione parecchio lenta nel caso in cui si aggiungano un certo numero di parametri.

L'**High Performance Computing (HPC)** indica l'insieme di **tecniche**

hardware e software utilizzate per risolvere problemi ad alta intensità computazionale, di memoria o di dati. Questa disciplina integra:

- architetture di **calcolo parallele**;
- **modelli di programmazione**;
- **algoritmi e metodi numerici**.

L'obiettivo è, dunque, quello di rendere risolvibili in tempi accettabili problemi che, di norma, non lo sarebbero in un computer normale. Le prestazioni in HPC dipendono dal corretto **bilanciamento tra hardware, software e algoritmi**. È bene notare che il calcolo ad alte prestazioni non coincide con l'uso esclusivo di supercomputer di grandi dimensioni, bensì riguarda il modo di utilizzare le risorse sfruttando **parallelismo e gerarchia di memoria**. Le stesse tecniche HPC sono applicate infatti anche a diversi tipi di sistemi che usiamo anche quotidianamente come CPU multicore, unità vettoriali **SIMD** o acceleratori con GPU.

Il classico modello di **Von Neumann** fa delle assunzioni che sono troppo limitanti:

- esecuzione di **una istruzione alla volta**;
- un **singolo flusso di controllo**;
- accesso a **una memoria uniforme**.

E, per quanto questo modello sia utile per descrivere algoritmi e dimostrare la loro complessità e correttezza, questa è una semplificazione troppo grossa rispetto all'hardware reale. Infatti, al giorno d'oggi il tempo di esecuzione di un programma è dominato dall'attesa dei dati in accesso dalla memoria e non dal calcolo in sé per sé. Inoltre, il modello sequenziale assume un accesso alla memoria uniforme e immediato, che non riflette la realtà.

Si può fare riferimento a due leggi:

1. **Legge di Moore**: il numero di transistor raddoppia ogni 18-24 mesi.
2. **Scaling di Dennard**: frequenza può aumentare mantenendo potenza e dissipazione costanti migliorando l'hardware.

Il risultato di queste due leggi è che abbiamo un aumento automatico delle prestazioni grazie a più transistor e, quindi, un' esecuzione più veloce del codice sequenziale. Tuttavia, nel 2006 lo scaling di Dennard si è arrestato e si arriva al punto in cui non si può fare meglio di così sfruttando solo codice sequenziale. È dunque necessario **sfruttare l'hardware in parallelo**

e sfruttare in modo efficiente la **gerarchia di memoria**. Per cui, e prestazioni diventano una responsabilità del programmatore e si rende necessario adottare il **performance engineering**.



L'hardware moderno è intrinsecamente parallelo e sfrutta sistemi **multi-core**, fa uso di **Hyper-Threading** cioè possono coesistere più thread hardware per core e sfrutta **Acceleratori GPU**. Per fruttare a pieno queste architetture, è necessario che il software sia **esplicitamente parallelo**. Esistono più livelli di parallelismo:

1. **Bit-level parallelism**: operazioni su parole di dimensione crescente;
2. **Instruction-level parallelism (ILP)**: pipeline ed esecuzione fuori ordine;
3. **Data-level parallelism**: operazioni vettoriali (SIMD) cioè che vengono effettuate contemporaneamente su dati diversi;
4. **Thread-level parallelism**: esecuzione concorrente di più thread in parallelo;
5. **Process-level parallelism**: esecuzione distribuita su più processi o nodi;

Ogni livello contribuisce alle prestazioni complessive e ignorarne anche solo uno porta a un uso non efficiente dell'hardware.

Un sistema HPC è tipicamente organizzato come un cluster di nodi di calcolo i cui nodi sono collegati tramite una rete ad alte prestazioni a **bassa latenza e alta banda**. I dati sono spesso gestiti tramite **storage condivisi** accessibili a tutti. Un cluster è un **insieme di nodi di calcolo indipendenti**, collegati tramite una rete ad alte prestazioni. Ogni nodo dispone di

una propria CPU, memoria e acceleratori e un sistema operativo completo. Il sistema è gestito tramite uno **stack software stratificato** che garantisce correttezza, prestazioni e scalabilità. Abbiamo diversi livelli dello stack:

- **Applicazioni:** che rappresentano i codici scientifici da eseguire.
- **Librerie:** che sono librerie che forniscono supporto al calcolo numerico e al parallelismo.
- **Runtime:** ciò che gestisce l'esecuzione parallela.
- **Scheduler:** che alloca le risorse e gestisce i job. Nel nostro caso è bene segnalare che sfrutteremo lo scheduler SLURM che rappresenta l'attuale standard del settore.
- **Compiler:** che traducono il codice in codice macchina ottimizzato per una specifica architettura.

Tra questi, un ruolo fondamentale vedremo che lo avrà lo **scheduler** che si occuperà di gestire le risorse del cluster. Così facendo, gli utenti non eseguono i programmi direttamente, bensì sottomettono i job allo scheduler che li gestirà e ordinerà a seconda di quelle che sono le sue policy e delle risorse che può allocare. Questo viene fatto per far sì che sia lo scheduler a garantire l'uso efficiente delle risorse e l'equità tra gli utenti. Come abbiamo già detto, dunque, non basterà scrivere codice corretto, abbiamo anche bisogno di **pensare in termini di parallelismo** e gestire esplicitamente dati, comunicazione e sincronizzazione.

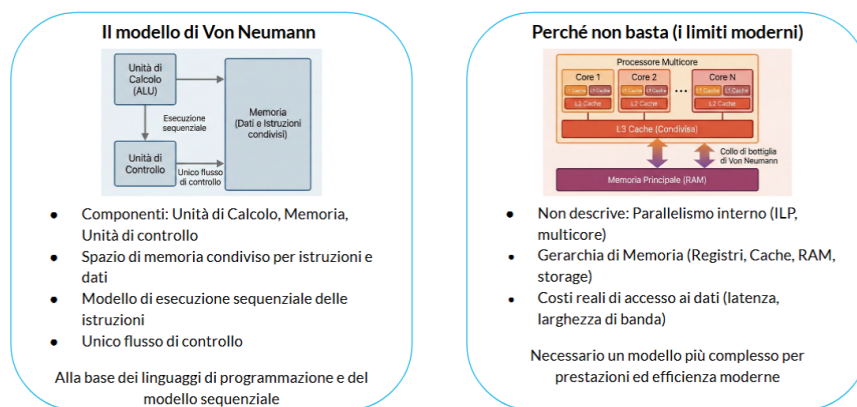
Capitolo 1

Architetture Moderne

Le prestazioni di un programma parallelo dipendono dall'architettura hardware e lo stesso codice può avere prestazioni molto diverse su architetture diverse. Ciò dipende dal fatto che l'architettura determina:

- Costi di accesso alla memoria.
- Disponibilità di parallelismo.
- Costo della computazionale e sincronizzazione.

Ignorare questi dettagli potrebbe comportare scelte inefficienti. Come già detto, il modello Von Neumann è piuttosto limitante rispetto a quella che è la realtà:



Infatti, la struttura di un core moderno è molto diversa rispetto a quella ideata da Von Neumann poiché troviamo:

- Unità di **Fetch/Decode**.

- Pipeline di esecuzione.
- Unità di esecuzione ALU, FPU e unità vettoriali.
- Registri e Cache di primo livello

1.1 Memory Wall

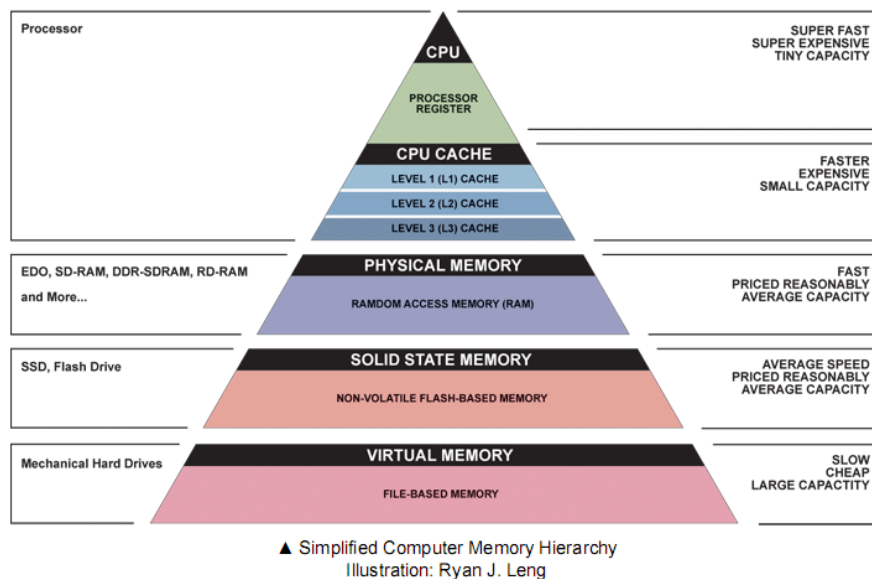
Il problema principale che dovremo risolvere è quello del **Memory Wall** per cui il calcolo della CPU sia diventato molto più veloce rispetto alla velocità di accesso alla memoria principale risultando così in sprechi di tempo dovuti al fatto che è necessario attendere l'accesso alla memoria. Per cercare di risolvere questo problema, utilizziamo due principi fondamentali:

- **Località temporale:** dati usati di recente vengono spesso riutilizzati.
- **Località spaziale:** dati vicini in memoria vengono utilizzati insieme.

Le architetture moderne sfruttano già i principi di località attraverso **cache**, **prefetching** e **gerarchia di memoria**.

1.1.1 Gerarchia di Memoria

I sistemi moderni usano una gerarchia di memoria a più livelli che segue, tipicamente, il seguente schema:



Ogni livello si differenzia dal precedente in termini di latenza, banda e capacità. Generalmente, minore è la capacità, minore sarà la latenza d'accesso.

Le memorie cache sono memorie molto veloci che memorizzano copie di dati presenti in memoria principale (seguendo il principio di località temporale) e i dati vengono trasferiti tra memoria e cache in **blocchi** (seguendo il principio di località spaziale). Abbiamo un **cache hit** quando il dato è presente in cache, mentre parliamo di **cache miss** quando il dato non è presente. A causa del costo elevato di accesso in memoria centrale, i cache miss hanno un costo estremamente elevato.

1.1.2 Prefetching

Il **prefetching** è un meccanismo che anticipa il caricamento dei dati e può essere effettuato tramite **hardware** o **software**. Questo può essere parecchio efficace quando abbiamo accessi ai dati regolari con pattern prevedibili ma svantaggio in caso di accessi indiretti o irregolari. Possiamo vedere, ad esempio, i seguenti due snippet di codice:

```
for (i = 0; i < N; i++)  
    sum += A[i];
```

```
for (i = 0; i < N; i++)  
    sum += A[index[i]];
```

Quello che vediamo nel secondo caso è che prima di accedere all'indice di A, abbiamo bisogno di accedere a un ulteriore indirizzo di memoria prima. Ciò rallenta chiaramente le prestazioni rispetto al primo caso.

1.2 Parallelismo Interno al Processore

Per aumentare il throughput senza aumentare la frequenza di clock, si sfruttano diverse tecniche di parallelismo.

1.2.1 Pipelining

La **pipeline** suddivide l'esecuzione di un'istruzione in più stadi:

- **Fetch**: prelevamento dell'istruzione.
- **Decode**: decodifica dell'istruzione.

- **Execute:** accesso ai registri ed esecuzione.
- **Memory Access:** accesso in memoria.
- **Write-back:** scrittura risultato sui registri.

Più istruzioni possono essere in esecuzione contemporaneamente in stadi diversi ciò con l'obiettivo di aumentare il throughput. Ci teniamo a sottolineare che il sistema di pipeline non migliora la latenza del sistema poiché questa è una caratteristica intrinseca della gestione della memoria del sistema. Vedremo più avanti come questo sistema di pipeline in realtà complichino il nostro sistema a causa di possibili stalli dovuti alle dipendenze tra istruzioni o anche a causa di errate guess nel possibile if che verrà eseguito.

1.2.2 Instruction-Level Parallelism (ILP)

L'**Instruction-Level Parallelism (ILP)** è il parallelismo tra istruzioni indipendenti. Un processore moderno sfrutta l'ILP potendo eseguire più istruzioni nello stesso ciclo di clock e riordinandole dinamicamente (out-of-order execution). Questo meccanismo è sfruttato tramite:

- **Pipeline multiple**
- **Unità di esecuzione multiple**

Il grado di ILP, sebbene sia una tecnica trasparente al programmatore, presenta limiti intrinseci dettati da: dipendenze tra le istruzioni, accessi alla memoria e struttura stessa del codice.

1.2.3 SIMD: Parallelismo Vettoriale

Il **SIMD (Single Instruction, Multiple Data)** rappresenta un set di istruzioni che permette l'applicazione della stessa operazione a più dati contemporaneamente. Le CPU moderne implementano questo paradigma includendo unità vettoriali dotate di registri di grande ampiezza. La vettorizzazione del codice può essere:

- **Automatica:** a carico del compilatore.
- **Esplicita:** a carico del programmatore. Sebbene molto efficace, la velocità ottenibile dal SIMD dipende strettamente dalla regolarità degli accessi in memoria e dall'assenza di dipendenze tra le iterazioni.

1.2.4 SMT: Simultaneous Multithreading

Il **SMT (Simultaneous Multithreading)** permette a un singolo core di eseguire più thread hardware contemporaneamente. In questo scenario, i thread condividono pipeline, unità di esecuzione e cache. Lo SMT mira a:

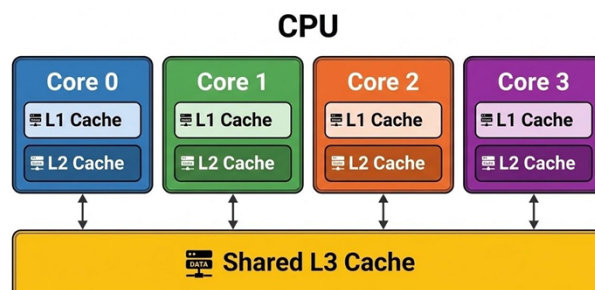
- Migliorare l'utilizzo delle risorse del core.
- Nascondere le latenze (come gli accessi in memoria).

È fondamentale notare che lo SMT non raddoppia le prestazioni, poiché i thread competono per le medesime risorse. La sua reale efficacia dipende dal tipo di carico di lavoro e dal comportamento della memoria. Sarebbe bene che thread hardware che condividono lo stesso core, utilizzino gli stessi dati così da evitare di poter sfruttare al meglio la cache L1 che condividono. Se non facessero a questo modo, si rischierebbe che i buttano fuori i dati dalla memoria a vicenda.

1.3 Sistemi Multicore e Modelli di Accesso alla Memoria

I **processori multicore** integrano più core di calcolo sullo stesso chip. Questa architettura rappresenta la principale risposta hardware alla fine dello scaling di Dennard e all'impossibilità fisica di continuare ad aumentare la frequenza di clock. In un sistema multicore:

- **Risorse private:** ogni core dispone di proprie pipeline, unità di esecuzione e cache (tipicamente L1 e L2).
- **Risorse condivise:** la cache di ultimo livello (L3) e la memoria principale sono condivise tra i core.



Le prestazioni finali di un processore multicore dipendono pesantemente dalla capacità del software di sfruttare più core in parallelo e di gestire in modo efficiente la condivisione delle risorse.

1.3.1 Il Modello MIMD

L'architettura **MIMD (Multiple Instruction, Multiple Data)** prevede che ogni unità di elaborazione esegua un flusso di istruzioni indipendente operando su dati differenti. Questo modello a parallelismo coarse-grained offre elevata flessibilità, è adatto a problemi eterogenei e rappresenta la base dei sistemi moderni.

1.3.2 SMP vs NUMA

Nei sistemi **SMP (Symmetric Multiprocessing)**, più processori condividono la stessa memoria fisica e lo stesso spazio di indirizzamento. Si basano su un modello **UMA (Uniform Memory Access)**, dove l'accesso alle risorse è paritario per tutti i processori ed equivalenti per il sistema operativo.

- **Vantaggi:** modello concettualmente semplice, comunicazione veloce intra-nodo e forte supporto da parte di OS e runtime.
- **Svantaggi:** scalabilità limitata (difficile andare oltre poche decine di core) a causa della contesa sul bus di memoria e del traffico hardware dovuto a protocolli necessari a mantenere la coerenza della cache.

Per superare i limiti di scalabilità dell'SMP, si utilizza l'**architettura NUMA (Non-Uniform Memory Access)**, in cui la memoria è fisicamente distribuita. Ogni gruppo di core possiede una memoria locale (con accesso più veloce), ma può accedere alla memoria remota (con accesso più lento).

I sistemi moderni sono spesso ccNUMA (cache-coherent NUMA): l'hardware mantiene la coerenza delle cache tra i vari nodi NUMA, offrendo al programmatore l'illusione di una memoria condivisa. Le prestazioni reali dipendono fortemente dalla località NUMA dei dati e dei thread.

1.4 Sistemi a Memoria Condivisa e Distribuita

In base al paradigma di accesso alla memoria, possiamo dividere i sistemi in due macro-categorie:

- **Sistemi a memoria condivisa:** Tutti i thread vedono lo stesso spazio di indirizzamento e comunicano implicitamente tra loro leggendo e scrivendo gli stessi dati. Il parallelismo è espresso tramite thread. La programmazione è generalmente più semplice, ma presenta svantaggi legati alla competizione per le risorse, la necessità di sincronizzazione e i costi di memoria non uniformi introdotti dalle architetture NUMA.
- **Sistemi a memoria distribuita:** Ogni processo dispone di una memoria privata e non esiste alcuno spazio di indirizzamento globale. La comunicazione tra processi avviene obbligatoriamente tramite lo scambio esplicito di messaggi. Questo approccio garantisce un'elevata scalabilità e rimuove la contesa diretta sulla memoria, al prezzo di una programmazione più complessa e di un costo esplicito (latenza) per ogni comunicazione.

1.4.1 Sistemi Ibridi e HPC

I sistemi ibridi costituiscono l'architettura standard per i moderni sistemi HPC:

- Sono composti da nodi multicore che operano internamente a memoria condivisa.
- Questi nodi sono poi collegati tra loro formando un cluster a memoria distribuita.

Di conseguenza, la gestione del parallelismo avviene su due livelli distinti e complementari:

- All'interno del singolo nodo (Intra-nodo): il parallelismo viene espresso tramite thread che accedono alla memoria condivisa locale (spessore regolato da architettura NUMA).
- Tra nodi diversi (Inter-nodo): il parallelismo è gestito tramite processi separati, e lo scambio di dati richiede una comunicazione esplicita (come l'invio di messaggi via rete).

1.4.2 Reti e Acceleratori

Nei sistemi HPC i nodi sono collegati tramite **reti ad alte prestazioni** e questa influenza la **latenza** della comunicazione, la **banda** disponibile e la **scalabilità** delle applicazioni. È anche possibile fare uso di **acceleratori** come dispositivi specializzati per il calcolo parallelo come la **GPU**.

1.5 Modelli di Analisi: Il Roofline Model

Il **Roofline model** è un modello prestazionale semplificato. Questo descrive il limite massimo di prestazioni di un'applicazione che è solitamente limitato a causa di capacità di calcolo o dalla banda di memoria. Questo modello mette in relazione **intensità computazionale** e **prestazioni ottenibili**. Esistono due tipi di sistemi:

- **memory bound.**
- **compute bound.**

E questo modello ci aiuta a capire dove intervenire per ottimizzare al variare di memoria messa a disposizione o potenza computazionale messa a disposizione.

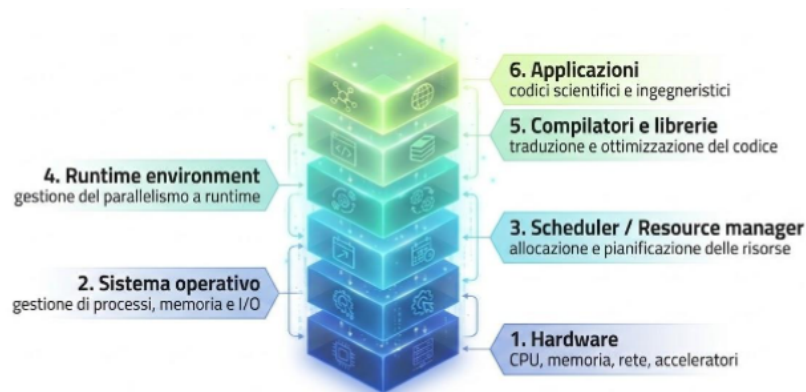
Capitolo 2

Software Stack in HPC

L'utilizzo di un cluster HPC differisce radicalmente da quello di un normale PC o di un server tradizionale. La necessità di introdurre uno stack software specifico nasce da tre fattori chiave:

- **La natura delle risorse:** I sistemi HPC mettono a disposizione risorse (CPU, memoria, rete e storage) che sono per definizione condivise, estremamente costose e spesso destinate a task critici.
- **La competizione:** A differenza di un PC personale, in un cluster ci sono costantemente molti utenti che competono per utilizzare queste stesse risorse nello stesso momento.
- **Il rischio del caos:** Senza un livello software in grado di orchestrare tutto questo, i risultati sarebbero disastrosi, portando a:
 - Spreco di risorse (es. nodi lasciati inattivi o sovraccaricati).
 - Conflitti tra utenti (programmi che si "rubano" memoria a vicenda o si bloccano).
 - Impossibilità di garantire le prestazioni previste e l'equità (fairness) nell'accesso al sistema da parte di tutti.

La regola d'oro è che **ogni livello interagisce con quelli adiacenti e influenza il comportamento prestazionale dell'intero sistema.**



Partendo dalle fondamenta fisiche e salendo fino all'utente, troviamo 6 livelli:

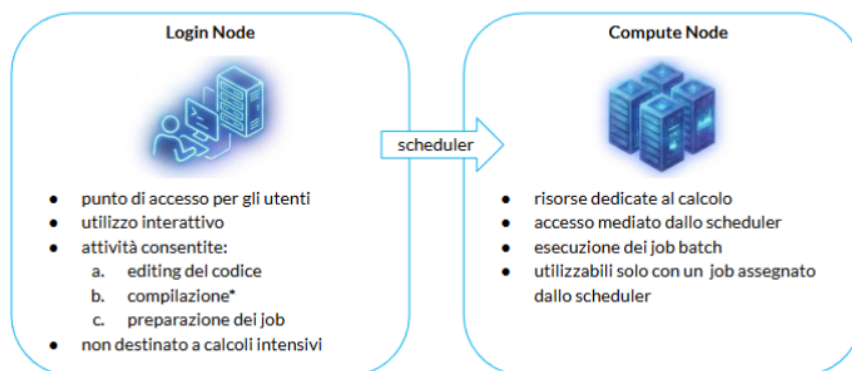
- **1. Hardware**: costituisce la base fisica (CPU, memoria, rete, acceleratori).
- **2. Sistema operativo**: si occupa della gestione di base dei processi, della memoria e dell'I/O.
- **3. Scheduler / Resource manager**: è il livello responsabile dell'allocazione e lo **scheduling** delle risorse del cluster.
- **4. Runtime environment**: gestisce il parallelismo a runtime durante l'esecuzione vera e propria.
- **5. Compilatori e librerie**: sono fondamentali per la traduzione e, soprattutto, l'ottimizzazione del codice affinché sfrutti l'hardware.
- **6. Applicazioni**: rappresentano i programmi finali, come i codici scientifici e ingegneristici degli utenti.

Dal punto di vista dell'utente, un sistema HPC non deve essere visto come una singola macchina, bensì come un vero e proprio **ambiente di calcolo controllato**.

Esso rappresenta, infatti, l'integrazione strutturata di hardware, software e specifiche **policy di utilizzo**. L'intera infrastruttura è stata pensata e progettata con il duplice scopo di eseguire **job di lunga durata** e di **massimizzare l'utilizzo complessivo** delle risorse a disposizione. Trattandosi di un sistema intrinsecamente **condiviso**, è importante tenere a mente che le risorse hardware non sono sempre immediatamente disponibili. Di conseguenza, l'accesso alla potenza di calcolo non avviene in modo diretto, ma è

sempre **mediato**: l'utente interagisce inizialmente solo con un nodo di accesso detto **nodo di Login** che si occuperà di gestire le richieste al cluster di calcolo vero e proprio.

Il **Login Node** funge da punto di accesso interattivo per gli utenti le attività consentite su questa macchina si limitano alla preparazione del lavoro, come l'editing del codice, la sua eventuale compilazione e l'impostazione dei job. È fondamentale ricordare che questo nodo **non è destinato all'esecuzione di calcoli**. Al contrario, i **Compute Node** sono le vere e proprie risorse dedicate al calcolo puro. L'accesso a questi nodi non è mai diretto, ma avviene sempre in modo mediato: essi si occupano dell'esecuzione dei job batch e sono utilizzabili esclusivamente in seguito all'assegnazione di un job da parte dello **scheduler**.



Questa rigida separazione non è casuale, ma rappresenta una scelta architetturale obbligata per gestire e proteggere un hardware estremamente costoso. Nello specifico, isolare le attività di preparazione (spesso interattive e imprevedibili) da quelle di calcolo serve a garantire la **stabilità** dell'intero sistema e a proteggere le risorse computazionali. Inoltre, questa barriera impedisce che le operazioni o gli errori di un singolo utente possano causare interferenze con i lavori in esecuzione degli altri, assicurando un utilizzo **equo ed efficiente** delle macchine e permettendo allo scheduler di effettuare una pianificazione controllata e ottimale dei job.

2.1 Filesystem nei sistemi HPC

La gestione dei dati all'interno dei sistemi HPC è affidata a un **filesystem condiviso** tra i login node e i compute node, garantendo così che i dati siano accessibili contemporaneamente da molti nodi. Questa infrastruttura è appositamente progettata per offrire un **elevato throughput** e per supportare

accessi paralleli, motivo per cui viene spesso implementata come un vero e proprio **filesystem parallelo**. A livello pratico, gli utenti interagiscono principalmente con due directory tipiche: la `$HOME`, dedicata alla conservazione dei dati personali e dei file di configurazione, e la `$SCRATCH`, pensata appositamente per ospitare i dati temporanei e per memorizzare l'output generato dai job in esecuzione.

Le modalità con cui le applicazioni effettuano le operazioni di Input/Output (I/O) hanno un impatto diretto e profondo sulle prestazioni e sulla scalabilità dell'intero sistema. Ad esempio, l'elaborazione di **molti file di piccole dimensioni** comporta un elevato overhead, il quale si traduce in una forte degradazione delle prestazioni globali. Analogamente, gli **accessi casuali** in memoria penalizzano sia il throughput che la latenza, non riuscendo a sfruttare l'ottimizzazione offerta dalla struttura del filesystem parallelo. Per poter gestire grandi volumi di dati in maniera efficiente, è strettamente necessario ricorrere all'**I/O parallelo**, che garantisce un accesso coordinato ai file da parte di molteplici processi contemporaneamente. In conclusione, la gestione dell'I/O rappresenta molto spesso il **vero limite alla scalabilità** di un programma, persino in quegli scenari in cui la componente di puro calcolo riesce a scalare in modo perfettamente corretto.

2.2 Compilatori in ambienti HPC

In un ambiente HPC sono tipicamente messi a disposizione degli utenti molteplici **compilatori**, tra cui troviamo le opzioni più note come GCC o Clang/LLVM, affiancate spesso da compilatori **vendor-specific** (come quelli forniti da Intel, AOCC o NVIDIA HPC). Il ruolo del compilatore va ben oltre la semplice traduzione del codice sorgente poiché si occupa di applicare **ottimizzazioni automatiche** e di generare un codice che sia specificamente ottimizzato per l'architettura su cui andrà in esecuzione. Per questo motivo, il compilatore non è un semplice strumento di traduzione, ma deve essere considerato a tutti gli effetti come **parte integrante delle prestazioni** finali di un'applicazione.

La ragione per cui all'interno di questi sistemi coesistono più compilatori è dovuta in primis alla grande diversità delle **architetture hardware** sottostanti. Ogni compilatore implementa, infatti, differenti **strategie di ottimizzazione** e gestisce in modo diverso i modelli di vettorizzazione e di parallelismo. Inoltre, è frequente che determinate librerie risultino maggiormente ottimizzate solo se trattate con compilatori specifici. Nella pratica, chi sviluppa si trova spesso a dover gestire una naturale **tensione** tra la massima portabilità del codice e la ricerca delle massime prestazioni. Proprio per

questo motivo, è fondamentale ricordare che **non esiste un compilatore "migliore" in assoluto**, ma bisogna saper scegliere quello più adatto al proprio caso d'uso e al proprio hardware.

È bene segnalare che si potrebbe provare a compilare il codice con più compilatori per vedere quale ci ritorna delle prestazioni migliori.

2.3 Librerie

Nei sistemi HPC, le **librerie** rappresentano dei componenti centrali dell'intero stack software, in quanto forniscono implementazioni già ampiamente ottimizzate per le operazioni fondamentali. Il loro grande vantaggio è quello di incapsulare al loro interno sia una profonda conoscenza dell'architettura hardware sia le ottimizzazioni più avanzate, tanto che spesso determinano le prestazioni finali molto più di quanto non faccia il codice applicativo stesso. Tra gli esempi più importanti a disposizione degli sviluppatori troviamo:

- **MPI**: fondamentale per gestire la comunicazione tra processi in sistemi a memoria distribuita.
- **OpenMP**: standard di fatto per gestire il parallelismo in architetture a memoria condivisa.
- **BLAS / LAPACK**: librerie di base, essenziali per le operazioni di algebra lineare.

Questa enorme ricchezza di strumenti, tuttavia, genera il cosiddetto **problema dell'ambiente software**. Rispetto a un computer classico, l'ambiente di un cluster HPC è intrinsecamente molto più complesso. Su queste macchine devono **necessariamente coesistere molteplici versioni di compilatori e innumerevoli versioni delle stesse librerie**, poiché le varie applicazioni degli utenti presentano dipendenze specifiche e requisiti spesso incompatibili tra loro. Un concetto fondamentale da tenere a mente è che il compilatore altera il risultato finale ad esempio una libreria MPI compilata con GCC è incompatibile e profondamente diversa da una compilata con strumenti Intel. Diventa quindi imperativo che l'ambiente sia sempre controllato, riproducibile e configurabile dinamicamente per ogni singolo utente e per ogni specifico job.

La soluzione standard adottata universalmente nei sistemi HPC per domare questa complessità prende il nome di **Environment Modules**. Si tratta di un meccanismo che permette di "caricare" dinamicamente solo le versioni specifiche del software necessario per il proprio lavoro, configurando

l'ambiente su misura. All'atto pratico, i moduli agiscono andando a modificare in modo sicuro le variabili d'ambiente del sistema operativo (come ad esempio i percorsi `PATH` o `LD_LIBRARY_PATH`). Questo approccio elegante garantisce la coesistenza pacifica di più compilatori e versioni dello stesso software, favorendo al contempo un elevato grado di controllo, grande flessibilità e, soprattutto, la totale riproducibilità degli esperimenti

2.4 Scheduler

Il **runtime environment** rappresenta il livello software responsabile della gestione e dell'esecuzione di un programma parallelo a tempo di run-time. Pur non facendo parte del kernel del sistema operativo (opera infatti in user-space al di sopra di esso), svolge compiti fondamentali come la **creazione e gestione dei thread** (es. tramite OpenMP), **l'inizializzazione e il coordinamento dei processi** (es. tramite MPI), e la **gestione della sincronizzazione e della comunicazione tra processi**. Inoltre, si occupa di applicare politiche cruciali per le prestazioni come il **CPU binding e la memory affinity**. Il comportamento di questo ambiente è pesantemente influenzato dalle variabili d'ambiente (come `OMP_NUM_THREADS`), dai parametri del job passati allo scheduler e dall'architettura fisica del nodo stesso.

Perché tutto questo ecosistema funzioni senza collassare, la presenza di uno **scheduler** è assolutamente indispensabile. In un ambiente in cui le risorse sono limitate, costose e condivise tra moltissimi utenti che effettuano richieste concorrenti e spesso incompatibili, operare senza questo componente porterebbe inevitabilmente al caos operativo, allo spreco di risorse e all'instabilità del sistema. Lo scheduler diviene quindi necessario per **pianificare l'uso delle risorse, evitare conflitti, garantire equità e massimizzare l'efficienza** complessiva del cluster.

Nello specifico, lo scheduler riceve le richieste di risorse (sotto forma di **job**) dagli utenti e agisce da vero e proprio arbitro:

- Decide autonomamente **quando** un job può effettivamente partire e **dove** (ovvero su quali nodi fisici) deve essere eseguito.
- **Assegna e isola** fisicamente le risorse necessarie per quel job, bloccando ad esempio un certo numero di CPU/core, un quantitativo specifico di memoria e le eventuali GPU richieste.
- Applica rigorosamente le **policy di sistema**, gestendo le priorità in coda, imponendo i limiti di tempo massimi consentiti per l'esecuzione e garantendo il *fair-share* (l'equità di utilizzo tra i vari utenti).

Attualmente, **Slurm** rappresenta lo scheduler standard di fatto che vedremo anche in questo corso.

2.5 Workflow per sistemi HPC

Nei sistemi HPC l'unità fondamentale di esecuzione non è il singolo programma o processo tradizionale, ma prende il nome di **job**. Un job può essere visto come un contenitore descrittivo che specifica allo scheduler esattamente **quali risorse** hardware sono necessarie, per **quanto tempo** lo saranno e **quale programma** dovrà essere effettivamente eseguito. Si tratta a tutti gli effetti di un'entità **schedulata**: una volta preparato, il job viene sottomesso allo scheduler, viene posizionato in una coda di attesa e verrà eseguito in totale autonomia solo nel momento in cui le risorse richieste si libereranno. La regola d'oro da ricordare è che in un ambiente HPC **non si esegue mai direttamente un programma, ma si richiedono le risorse per poterlo fare**.

All'atto pratico, l'interazione quotidiana dell'utente con il cluster segue un ciclo ben definito, noto come **workflow tipico HPC**, che viene ripetuto innumerevoli volte durante le fasi di sviluppo:

1. **Accesso al sistema**: si effettua il login interattivo esclusivamente sul login node.
2. **Preparazione dell'ambiente**: si caricano i moduli software (compilatori, librerie) necessari al proprio lavoro.
3. **Sviluppo e compilazione**: si scrive il codice sorgente e lo si compila utilizzando gli strumenti scelti al passo precedente.
4. **Preparazione del job**: si definiscono le risorse hardware richieste e si crea materialmente lo "script di job".
5. **Sottomissione del job**: si invia lo script appena creato allo scheduler affinché venga preso in carico.
6. **Esecuzione**: lo scheduler fa girare il job sui compute node non appena le macchine sono pronte.
7. **Analisi dei risultati**: una volta terminato il calcolo, si analizzano i file di output e di log generati per valutare i risultati o per fare eventuale debugging.

Alcuni errori da evitare assolutamente sono:

- Eseguire calcoli sul login node.
- Bypassare lo scheduler tramite SSH.
- Richiedere troppe o troppe poche risorse.
- Non considerare che l'ambiente non è quello standard di un normale PC.

Capitolo 3

Ottimizzazione

L'High Performance Computing richiede, come suggerisce il nome stesso, di ottenere la massima efficacia dal codice sulla macchina su cui viene eseguito. L'ottimizzazione in HPC non è un'operazione singola, ma un bilanciamento di diversi vincoli: tempo di esecuzione, uso della memoria, efficienza energetica, robustezza e costi di I/O.

3.1 Misurazione delle performance

Quando parliamo di performance cerchiamo di andare a fare una stima di quanto il nostro codice riesca a sfruttare delle specifiche risorse della CPU. In particolare, ci concentriamo su:

- Quanto è **veloce** il nostro codice.
- Quanto bene sfrutta il parallelismo a livello di istruzione.

3.1.1 Misure sul clock

Per quanto riguarda la velocità, sappiamo che la CPU non lavora in maniera continua ma lavora seguendo **cicli di clock** il cui ritmo è di parecchi clock al secondo. La frequenza può anche variare a seconda del carico di lavoro aumentando o diminuendo per risparmiare energia. Nello specifico, ci riferiremo a questo fenomeno come **throttling**.

Proprio perché il ciclo di clock varia, misurare semplicemente il tempo di esecuzione potrebbe portare a stime imprecise. Può essere più comodo andare a vedere **quanti cicli di clock sono stati spesi** su una certa sezione di codice. In particolare, ci concentriamo su sezioni di codice che ripetono

blocchi d'istruzioni su un array che rappresentano i punti più pesanti computazionalmente del nostro software. Questa metrica prende il nome di **Cycles Per Element** o **CPE**. Se vogliamo, il CPE rappresenta il numero di cicli di clock necessari all'analisi di una singola porzione di dato come, ad esempio, un singolo elemento dell'array.

Al contrario, nel caso volessimo stimare quanto bene stiamo sfruttando la capacità super-scalare della CPU, ha più senso usare la matrice di **Istruzioni per Ciclo (IPC)** per valutare quante istruzioni per ciclo vengono eseguite. Questo ci darà l'informazione riguardo a quante operazioni stiamo riuscendo a fare contemporaneamente all'interno di un ciclo di clock.

3.1.2 Misure sul Tempo

Nonostante le misure sul clock siano sicuramente molto utili e precise, spesso si preferisce andare a ragionare in termini di **Tempo di esecuzione**. Fondamentalmente, guarderemo tre diversi tipi di tempo:

- **Wall-clock time**: Fondamentalmente questo è il tempo d'orologio dall'inizio alla fine del processo.
- **System-Time**: quantità di tempo che l'intero sistema trascorre eseguendo il codice. Questo comprende il tempo in **kernel mode**.
- **Process user-Time**: Metrica che tiene il tempo speso dalla CPU per eseguire le istruzioni del codice eliminando, in pratica, i tempi morti d'attesa. Questo è il tempo in **user mode**
- **CPU time**: Somma di system time e process user time.

Indipendentemente dalle metriche che stiamo accumulando, affrontare solo una misura non è una buona stima: bisogna fare la media su molte esecuzioni e sottrarre l'overhead della funzione di tempo stessa.

Per misurare questi tempi in C possiamo utilizzare principalmente funzioni come *gettimeofday()* che restituisce il wall clock time, *clock()* che restituisce la somma tra lo user time e il system time e, infine, *clock_gettime()* che ritorna vari tipi di tempi tra cui real time, monotonic ecc.

```
#define CPU_TIME (clock_gettime( CLOCK_PROCESS_CPUTIME_ID, &ts ),\
                  (double)ts.tv_sec + \
                  (double)ts.tv_nsec * 1e-9)

....

Tstart = CPU_TIME ;
// your code segment here

Time = CPU_TIME - Tstart;
```

Come si vede in figura, è possibile misurare il tempo effettuato in una sezione di codice incapsulandolo tra i due CPU time.

Un'altra cosa da tenere in considerazione è che il nostro processore, al giorno d'oggi, è **multi-core** e, spesso, i processi possono spostarsi tra un core e l'altro. Di fatto ciò comporta dei rallentamenti a causa del fatto che è necessario nuovamente riprendere i dati in memoria e metterli nella cache allocata per ogni singolo core (vedremo dopo in dettaglio). Per evitare questo, si usano delle funzioni particolare che effettuano un binding di un processo ad uno o più core.

3.1.3 Profiling

Il **profiling** è una fase cruciale del *Performance Engineering*. Consiste nell'analizzare l'esecuzione di un programma utilizzando carichi di lavoro (*workload*) reali per identificare i **punti critici** (*hotspots*) e i **colli di bottiglia** del codice.

Invece di procedere per tentativi, il profiling permette di guidare il processo di ottimizzazione basandosi su dati oggettivi. Un'analisi accurata permette di individuare:

- **Intensità di calcolo e dati:** identifica quali specifiche linee di codice eseguono il maggior numero di operazioni o richiedono lo spostamento di grandi volumi di dati.
- **Stalli delle risorse:** rileva i punti in cui la CPU è ferma in attesa (ad esempio a causa di *pipeline stalls* o latenze della memoria).
- **Efficienza dell'I/O:** traccia il tempo speso nelle operazioni di input/output, evidenziando schemi di accesso al disco inefficienti.
- **Bilanciamento del carico (Workload balance):** fondamentale in ambito parallelo, serve a capire se il lavoro è distribuito equamente tra i core o se ci sono processi che rallentano l'intera esecuzione.

Questa pratica la vedremo dopo nel dettaglio.

3.2 Ottimizzare

Come abbiamo detto, ottimizzare vuol dire **estrarre la massima efficacia del codice sulla macchina su cui viene eseguito**. È importante notare come sottolineiamo il fatto che questo processo sia riferito a una specifica

macchina questo perché, a causa di dimensioni di memoria e simili, alcune scelte prese potrebbero essere ottimali su una macchina ma non sull'altra.

Inoltre, è necessario definire su cosa vogliamo ottimizzare. Questo perché potrebbe non essere possibile ottimizzare un codice a tutto tondo, ma potrebbe essere necessario dover trovare compromessi. Tra le varie condizioni di cui tener conto quando vogliamo ottimizzare, troviamo:

- Vincoli di tempo;
- Vincoli di memoria;
- Vincoli di I/O;
- Desiderio di robustezza del sistema;
- Limiti energetici.

Per quanto adesso il nostro focus sia incentrato sull'ottimizzazione del codice, è bene ricordare che prima di tutto dobbiamo:

- Verificare la **correttezza** del codice. Cioè che faccia già correttamente quello che deve fare.
- Usare il giusto **modello di dati** in termini di struttura dati ecc.
- Scegliere il giusto **algoritmo**.
- Avere un **buon codice** che sia chiaro, pulito, conciso e documentato. Un codice sporco è difficile da ottimizzare perché nasconde le inefficienze. Usare il principio **DRY (Don't Repeat Yourself)** evita il debito tecnico e facilita il debug. Inoltre, è bene che il codice sia **commentato**.
- Un'altra cosa da tenere in considerazione è il **Design** che comprende, in genere:
 - Teoria, quindi lettura di come si svolgono solitamente le cose.
 - Analisi dei vincoli e delle risorse che si hanno a disposizione.
 - Traduzione della teoria in codice computabile.
 - Testing che consiste nel verificare che tutto il codice o specifiche sezioni funzionino come ci si aspetta.
 - Validazione per assicurarci che il codice faccia ciò che deve fare e verificare per assicurarci che il codice faccia ciò che fa correttamente.

Un codice meravigliosamente ottimizzato che adotta l'algoritmo sbagliato è un controsenso, perché un algoritmo migliore, anche se implementato male, batterà (quasi) sempre il codice precedente per un numero di casi sufficientemente ampio.

In sintesi dovremmo lavorare come segue:

1. Scrivere un programma che ci dia le risposte corrette e che si comporti correttamente sempre.
2. Adottare gli algoritmi e le strutture dati più adatti ai nostri vincoli.
3. Avere un codice sorgente pulito e ottimizzabile dal compilatore.
4. Ottenere un flusso di lavoro basato sui dati e sui compiti.
5. Verificare il codice in diverse condizioni anche non direttamente dipendenti dal codice stesso come i vari carichi di lavoro ecc. per trovare i colli di bottiglia e le inefficienze.
6. Applicare tecniche di ottimizzazioni modificando i punti critici nel codice per rimuovere i blocchi di ottimizzazione.
7. Tornare a 1 se necessario.

Il **compilatore** è un programma che traduce il linguaggio sorgente in un linguaggio target equivalente. Un **interprete** è un programma di elaborazione del linguaggio che esegue direttamente un programma in un **linguaggio sorgente**. La compilazione fa parte di una **pipeline** che traduce il codice sorgente in un codice eseguibile dalla macchina. Questa pipeline comprende:

- **Preprocessore**: Analizza il sorgente includendo gli header ed espandendo le macro.
- **Front-end**: Traduce il sorgente in un linguaggio ad alto livello di astrazione specifico. Spesso assembly.
- **Assembler**: Produce il codice macchina dall'assembly generato.
- **Linker**: Risolve i riferimenti e gli indirizzi di memoria tra diverse sezioni del codice e librerie esterne.

I compilatori sono anche in grado di eseguire un'analisi sofisticata del codice sorgente in modo da produrre un codice target (solitamente un codice assembly) altamente ottimizzato per una data architettura target. Quindi, per prima cosa, lasciamo fare il lavoro al compilatore. Esistono diversi livelli di ottimizzazione generati automaticamente dal compilatore mediante specifico flag **-On**:

- `-O0`: Nessuna ottimizzazione (default), utile per il debug.
- `-O1`, `-O2`, `-O3`: Livelli crescenti di aggressività. **Nota bene**: non è detto che `-O3`, sebbene spesso generi un codice più veloce, sia ciò di cui hai realmente bisogno; su alcune architetture può generare codice troppo voluminoso che satura le cache, rendendo preferibile `-Os` (ottimizzazione per dimensione).
- `-march=native`: Istruisce il compilatore a generare codice specifico per l'architettura della CPU su cui sta girando, sfruttando tutti i set di istruzioni (ISA) disponibili. Infatti, il compilatore generalmente produce un codice portabile, cioè un codice che può essere eseguito su qualsiasi CPU appartenente a quella classe di architettura.

Esistono anche diversi modelli tipici di compilazione come, ad esempio **fbrach-probabilities** o **fprofile-generate**.

Altra cosa utile che approfondiremo maggiormente anche dopo è l'uso di **memoria contigua** che, in quanto contigua, evita continui salti da un punto di memoria a un altro.

Esistono anche diverse classi di storage:

- `extern`: variabili globali che esistono per sempre.
- `auto`: variabili locali allocate sullo stack.
- `register`: variabili che suggeriamo al compilatore di tenere in registro perché magari molto usata.

Inoltre, esistono anche dei quantificatori di variabili:

- **const**: indica che il valore dentro la variabile non cambia mai.
- **volatile**: indica che la variabile è modificabile dall'esterno del programma.
- **restrict**: indica che l'indirizzo di memoria è accessibile solo tramite specifico puntatore.

3.2.1 Memory Aliasing

Il **Memory Aliasing** si verifica quando due o più riferimenti (puntatori) puntano alla stessa posizione di memoria. Questo è uno dei maggiori blocchi alle prestazioni in C/C++.

```

void func1 ( int *a, int *b ) {
    *a += *b;    -> *a e *b adesso contengono 2
    *a += *b;    -> *a e *b adesso contengono 4
}

void func2 ( int *a, int *b )
    *a += 2 * *b; -> *a e *b adesso contengono 3
}

```

Consideriamo queste due funzioni. In effetti, se *a* e *b* puntano a due indirizzi diversi, il risultato delle due funzioni è lo stesso. Il discorso cambia, invece, quando *a* e *b* puntano allo stesso indirizzo come mostrato in figura.

Se il compilatore non può garantire che due puntatori siano distinti, non può ottimizzare gli accessi alla memoria perché teme che una scrittura su uno possa influenzare il valore dell'altro. Questo costringe la CPU a ricaricare continuamente i dati dalla memoria centrale anziché usare i registri o la cache. Per risolvere questo problema usiamo il qualificatore *restrict*.

In generale è utile:

- Scrivere codice evitando aliasing di memori rendendo chiaro a cosa serve una variabile e quando e mantenendo sotto controllo i branch condizionali.
- Usare strutture dati buone cache-friendly facendo sì che dati che vengono usati insieme, siano insieme. Inoltre, è necessario evitare il **false-sharing** che accade quando due thread lavorano su variabili diverse che però sfortunatamente risiedono nella stessa linea di cache, costringendo i core a invalidarsi la memoria a vicenda continuamente.
- Avere un buon flusso di lavoro così che il compilatore possa ottimizzare i branch e gli accessi alla memoria.
- Sfruttare le architetture multi-core superscalari e out of order.

Capitolo 4

Ottimizzazione Single-Core

Prima di tutto è bene iniziare dal punto più piccolo che si occupa di eseguire le nostre istruzioni cioè il **core**. Infatti, sebbene attualmente le CPU siano multi core, è importante cercare di capire come sfruttare ogni singolo core al meglio per tirare fuori il massimo potenziale da ogni risorsa a nostra disposizione.

Le prestazioni del software moderno dipendono dalla comprensione del divario tra l'architettura ideale di Von Neumann e la complessità dei core attuali. Mentre il modello classico prevede una memoria "piatta" con costi di accesso uniformi, le architetture moderne sono caratterizzate da gerarchie profonde e parallelismo interno per contrastare il **Data Starvation** ossia la mancanza dei dati necessari al lavoro attualmente in corso.

4.1 Memoria cache

Come già detto, a partire dai primi anni '90, la velocità delle CPU è cresciuta molto rapidamente rispetto a quella della RAM provocando un collo di bottiglia noto come **Memory Wall** in cui la CPU è costretta ad aspettare i tempi di accesso in memoria per continuare la sua elaborazione.

Per risolvere questo problema, si utilizza una **gerarchia di memoria** con costi in termini di cicli di clock crescenti:

1. **Registri:** questa è banalmente la memoria che utilizza il singolo core per effettuare le proprie operazioni e, ogni elemento su cui deve essere fatta un'operazione, deve passare da qui.
2. **Livello 1:** questa è una memoria cache presente all'interno di ogni singolo core e il suo accesso richiede circa 2 cicli di clock.
3. **Livello 2:** questa è una cache che può appartenere a uno o a più core e per effettuare l'accesso necessita di 10 cicli di clock.

4. **Livello 3:** infine troviamo una cache di terzo livello che è all'interno della CPU ed è condivisa da tutti i core.
5. **RAM:** La RAM è l'ultima "speranza" necessaria a recuperare le informazioni se non presenti in cache.

L'hit rate della memoria cache comporta grossi cambiamenti in termini di efficienza del sistema poiché basti pensare che passando dal 97% al 100% si passa dall'avere fino al 50% o 150% di prestazioni in più. Se volessimo fare i conti, dovremmo usare questa formula:

$$L1_{cost} * L1_{hitrate} + L1_{missrate} * RAM_{cost} = Cycles$$

Definiamo dei dati **locali** come dei dati che risiedono in una piccola porzione dello spazio degli indirizzi acceduta in un breve periodo di tempo. Da qui definiamo i due principi di località:

- **Località temporale:** dati referenziati di recente vengono spesso riutilizzati.
- **Località spaziale:** dati vicini in memoria vengono spesso referenziati insieme.

Da qui possiamo definire alcuni tipi di miss che possono andare a degradare le prestazioni della cache:

1. **Compulsory:** accessi inevitabili alla prima referenziazione a un dato.
2. **Capacità:** cache troppo piccola per contenere i dati necessari al lavoro.
3. **Conflitto:** svuotamento della cache dovuto alla mappatura dei dati sulle stesse linee di cache necessari per scambi di dati.

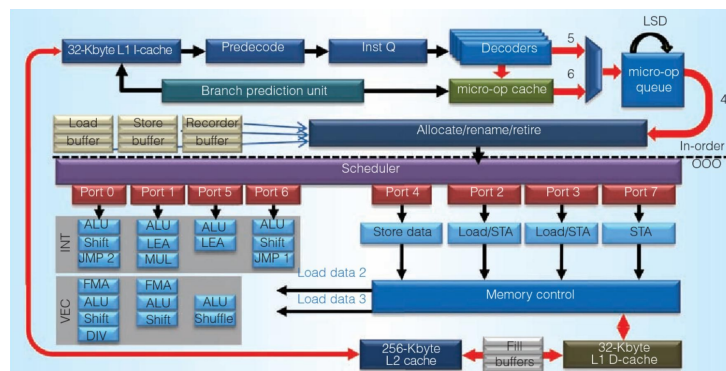
A questo punto, possiamo andare a dedurre anche quelli che sono dei modi per migliorare le prestazioni, banalmente, bilanciando i problemi di cui abbiamo parlato prima:

1. **Riorganizzare (codice e dati):** progettare il layout per migliorare la località temporale e spaziale.
2. **Ridurre (dimensione):** dimensione dei dati più piccola, meno istruzioni e per essere più facilmente contenuta in cache.
3. **Riutilizzare (linee di cache):** aumentare la località per mantenere i dati residenti per più operazioni.

Come abbiamo detto prima, sappiamo che le CPUCore, al giorno d'oggi contengono delle memorie cache "locali". Questo comporta il problema che se due cache di due core diversi contengono lo stesso dato, se uno accede in scrittura potrebbe modificare il dato senza che l'altro se ne renda conto. Questo problema comporta un certo costo difficoltà per essere gestito correttamente.

4.2 CPU

A metà degli anni '90 le CPU abbiamo già detto che diventano molto più veloci della memoria. Questo è dovuto principalmente all'uso di **CPU superscalari**:



La capacità superscalare di questo tipo di CPU è data dall'avere diverse **porte di accesso** per mandare in esecuzione le istruzioni e diverse unità logiche per eseguirle. Notiamo una divisione in:

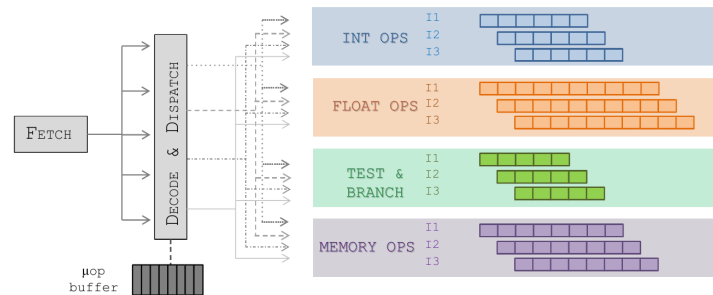
- **FrontEnd:** è la parte che recupera le istruzioni dalle cache le decodifica, predice i branch e smista le istruzioni sulle diverse porte.
- **BackEnd:** è la parte responsabile dell'esecuzione effettiva delle istruzioni e della riscrittura in memoria nel caso in cui sia necessario. Questa parte si occupa anche di gestire l'esecuzione delle operazioni **out-of-order** a seconda delle dipendenze che possono esserci.

4.2.1 Pipeline di Esecuzione

Le operazioni effettuate dalla nostra CPU seguono in realtà una serie di passaggi ben definiti:

- **Fetch:** fase in cui vengono richiamate le istruzioni dalla memoria/cache.
- **Decode:** fase in cui l'istruzione è interpretata e compresa.
- **Esecuzione:** in cui l'operazione viene eseguita.
- **Writeback:** fase in cui il risultato deve essere registrato in memoria.

Queste operazioni possono essere eseguite da una singola istruzione per volta e, chiaramente richiedono un certo tempo per essere eseguite. Per evitare di dover aspettare che ogni singola istruzione finisca la Pipeline, facciamo sì che ogni istruzione sia in un certa fase diversa della pipeline, così da per aumentare il **throughput**. Un altro modo per ottimizzare la pipeline è quello di utilizzare **pipeline multiple** che si dividono in base al tipo di istruzione che è necessario effettuare. Infatti, abbiamo detto che esistono varie unità di esecuzione che sono indipendenti tra di loro e che, quindi, possono lavorare in parallelo.



Il pipelining è sicuramente ottimo quando le istruzioni sono tutte indipendenti l'una dalle altre, tuttavia introducono alcuni problemi:

1. **Data hazard:** un'istruzione dipende dal risultato di un'istruzione precedente che non ha ancora completato la pipeline. **Soluzione:** La CPU deve inserire stalli (bubble) nella pipeline, perdendo cicli produttivi.
2. **Control hazard:** quando la CPU incontra un branch condizionale, non sa quale ramo eseguirà finché non valuta la condizione, ma nel frattempo la pipeline ha già iniziato a fare il prefetch delle istruzioni successive. **Soluzione:** branch predictor, un componente hardware che cerca di indovinare quale ramo verrà preso, basandosi sulla storia dei branch precedenti.

Infine, un'ulteriore ottimizzazione viene introdotta alla fine degli anni '90 con i **registri vettoriali**. Questi sono grandi registri nella CPU che permettono di eseguire una singola istruzione su tutti gli elementi del registro vettoriale in parallelo. Infine, all'inizio degli anni 2000 troviamo che i sistemi sono dotati di più core avvicinandoci quindi alla struttura più comune a oggi.

4.3 Evitare le Inefficienze

Scrivere codice pulito e logico è il prerequisito fondamentale per scrivere del buon codice, tuttavia, per ottenere le massime prestazioni da esso è necessario evitare inefficienze legate a salti, pipeline, cicli e controlli condizionali. Esistono cinque regole pratiche da applicare durante la stesura del codice.

4.3.1 Esempio che seguiremo

Supporremo di avere una distribuzione di punti casuali in un punto 2D che è diviso in sottoregioni. Per ogni punto, vogliamo selezionare tutte le celle della griglia il cui centro è più vicino a p di un certo raggio r ed eseguire su di esse alcune operazioni. Partiamo, dunque, da questo codice:

```
for (p = 0; p < Np; p++) {
    for (i = 0; i < Ng; i++) {
        for (j = 0; j < Ng; j++) {
            for (k = 0; k < Ng; k++) {
                dist = sqrt(
                    pow(x[p] - (double)i/Ng + half_size, 2) +
                    pow(y[p] - (double)j/Ng + half_size, 2) +
                    pow(z[p] - (double)k/Ng + half_size, 2)
                );
                if (dist < R) {
                    fai_qualcosa;
                }
            }
        }
    }
}
```

4.3.2 1. Evitare Chiamate a Funzioni Costose

Alcune chiamate a funzioni, sono parecchie costose anche a causa del doversi collegare a delle librerie esterne. In questo codice, ad esempio, poiché calcoliamo la distanza tra vari punti, troviamo la funzione *pow* e la funzione

sqrt per fare il quadrato e la radice. Notiamo che possiamo anche evitare questa cosa poiché ci basta considerare la distanza al quadrato dato che, alla fine, ci basta che vengano presi quegli specifici punti. Inoltre, sappiamo che i computer non sono molto bravi a effettuare le operazioni di divisione poiché hanno la necessità di calcolare l'inverso. Per evitare questo, possiamo precalcolarlo se è necessario effettuare la divisione più volte. Infine, poiché $(double)i * Ng_inv + half_size$ è indipendente dal punto preso, possiamo anche questo, precalcolarlo in un array.

```
for (i = 0; i < Ng; i++) {
    for (j = 0; j < Ng; j++) {
        for (k = 0; k < Ng; k++) {
            dx = x[p] - (double)i * Ng_inv + half_size;
            dy = y[p] - (double)j * Ng_inv + half_size;
            dz = z[p] - (double)k * Ng_inv + half_size;

            dist2 = dx*dx + dy*dy + dz*dz;
            if (dist2 < R2) {
                /* fai qualcosa */
            }
        }
    }
}
```

Nella versione presenta, ancora non è stata ancora inserita l'ultima ottimizzazione.

4.3.3 2. Hoisting delle espressioni

Guardando il codice precedente, possiamo soffermarci su alcuni specifici calcoli. Infatti, nel ciclo più interno noi calcoliamo dx e dy senza che questi dipendano effettivamente da k . Per quanto questo possa sembrare semplicemente una scelta per non rendere il codice confusionario, in realtà provoca grossi ritardi a causa del fatto che questi vengono calcolati k volte più del necessario. Possiamo dunque spostarli nel ciclo più in alto così da evitare questo problema.

```
double ijk[Ng];
for (i = 0; i < Ng; i++) {
```

```

    ijk[i] = (double)i * Ng_inv + half_size; // Pre-calcolo
}

for (i = 0; i < Ng; i++) {
    dx2 = x[p] - ijk[i];
    dx2 = dx2 * dx2; // Spostato nel ciclo 'i'

    for (j = 0; j < Ng; j++) {
        dy2 = y[p] - ijk[j];
        dy2 = dy2 * dy2;
        dist2_xy = dx2 + dy2; // Spostato nel ciclo 'j'

        for (k = 0; k < Ng; k++) {
            dz = z[p] - ijk[k];
            dist2 = dist2_xy + dz * dz; // Solo questo in k

            if (dist2 < Rmax2) { /* fai qualcosa */ }
        }
    }
}

```

4.3.4 3. Chiarire lo Scope delle Variabili

Nell'esempio considerato, troviamo delle variabili che sono funzionali alla specifica operazione che stiamo facendo. In particolare ci riferiamo a dx , dy , dz e $dist$. Una cosa che possiamo fare è andare a definirle all'interno stesso del ciclo in cui vengono utilizzate. Questo aiuta di molto il compilatore che sa con certezza che non è necessario conservarne il valore al di fuori di quelle parentesi.

```

for(int i = 0; i < Ng; i++) {
    // dx2 vive solo all'interno del ciclo 'i'
    double dx2 = x[p] - (double)i * Ng_inv + half_size;
    dx2 *= dx2;

    for(int j = 0; j < Ng; j++) {
        // dy2 e dist2_xy vivono solo qui
        double dy2 = y[p] - (double)j * Ng_inv + half_size;
        double dist2_xy = dx2 + dy2 * dy2;

        for(int k = 0; k < Ng; k++) {
            // dz e dist2 vivono solo qui

```

```
double dz = z[p] - (double)k * Ng_inv + half_size;
double dist2 = dist2_xy + dz * dz;

if (dist2 < Rmax2) {
    // fai qualcosa con sqrt(dist2);
}
}
}
```

4.3.5 4. Suggerisci cosa è Importante

Quando una variabile viene calcolata e riutilizzata frequentemente all'interno di un loop critico, può essere utile suggerire al compilatore di dedicarle un registro hardware utilizzando la parola chiave `register`. È fondamentale ricordare che si tratta solo di un **suggerimento non vincolante**: il compilatore, dopo aver analizzato il codice, potrebbe comunque decidere una strategia di allocazione diversa a causa di ragionamenti interni suoi sul come avrà intenzione di gestire la pipeline.

4.3.6 5. Non ripetere controlli o referenziazioni non necessarie

Spesso, anche a causa delle nostre abitudini di lavoro, ci ritroviamo spesso a ripetere dei controlli che spesso sono ridondanti rispetto a quello che già controlliamo. Infatti, questo è dovuto al fatto che la maggior parte delle volte siamo convinti di fare un bene al nostro codice rendendolo più sicuro quando, in realtà, non stiamo facendo altro che rallentarlo. Vediamo un esempio:

```
char * find_char_in_string( char *string, char c )
{
    int i = 0;
    // strlen() viene chiamata a ogni iterazione
    while ( i < strlen(string) ) {
        if ( string[i] == c )
            break;
        else
            i++;
    }
    if ( i < strlen(string) )
```

```
        return &string[i];
    else
        return NULL;
}
```

In questo caso, il problema di performance è causato dal fatto che `strlen` attraversa l'intera stringa a ogni ciclo per calcolarne la lunghezza, portando a una complessità quadratica. La stringa, ovviamente, non cambia nel corso delle iterazioni, dunque, ha senso calcolarla prima così da non ripetere una chiamata a funzione esterna. Esistono due soluzioni parecchio interessanti:

```
//V.1 La lunghezza è pre-calcolata
int i = 0;
int len = strlen(string);
while (i < len) {
    if(string[i] == c) break;
    else i++;
}

//V.2 Controllo del terminatore tramite puntatori
char *pos = string;
while((*pos != '\0') && (*pos != c))
    pos++;
return (*pos == '\0'?NULL:pos);
```

Simile è quello che accade con questa funzione riguardo alla referenziazione della memoria:

```
// Accesso in memoria ad ogni iterazione
void reduce_vector(int n, double *array, double *sum) {
    for (int i = 0; i < n; i++) *sum += array[i];
    // Scrive e legge l'indirizzo di 'sum' N volte
}
```

L'introduzione di un accumulatore locale separato permette di sfruttare i registri veloci della CPU, limitando la scrittura in memoria a una sola istruzione una volta terminato il ciclo:

```
// Uso di un accumulatore locale
void reduce_vector(int n, double *array, double *sum) {
    double temp = 0; // Variabile locale
    for (int i = 0; i < n; i++) {
        temp += array[i]; // Elaborazione veloce e locale
    }
    *sum = temp; // Un solo accesso in memoria
}
```

4.3.7 Avvertenza su Numeri in Virgola

Si potrebbe pensare che un compilatore con un alto livello di ottimizzazione (es. `-O3`) sia sempre in grado di riorganizzare i calcoli matematici per renderli più convenienti ma **questo non è vero per i numeri in virgola mobile**.

Mentre l'aritmetica intera in complemento a 2 è commutativa e associativa, la matematica in virgola mobile non è mai associativa:

$$(a + b) + c \neq a + (b + c) \quad (4.1)$$

A causa del numero limitato di cifre (precisione finita), l'ordine delle operazioni altera il risultato finale. Ad esempio, considerando una precisione limitata, sommare $1.00 + 0.001$ restituisce 1.00 , causando una perdita del decimale che invece non si avrebbe sommando prima i numeri più piccoli. Di conseguenza, il compilatore **non rimescolerà mai l'ordine delle operazioni floating point**, costringendo lo sviluppatore a scrivere direttamente la forma algebrica più efficiente nel codice sorgente.

4.4 Ottimizzazione basata sulla Cache

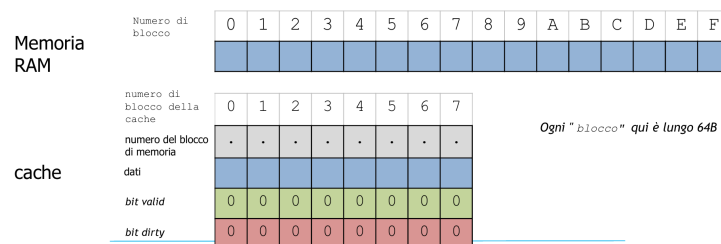
Abbiamo già visto la cache e il motivo della sua introduzione per risolvere il problema del **Memory Wall** e abbiamo già parlato della **gerarchia di memoria** che questa introduce. Per cercare di capire come ottimizzare al meglio l'utilizzo della cache ripetiamo adesso i due principi fondamentali:

- **Località temporale:** dati referenziati di recente vengono spesso riutilizzati.
- **Località spaziale:** dati vicini in memoria vengono spesso referenziati insieme.

Teniamo bene a mente questi due principi per capire comprendere al meglio le scelte che vengono prese nella mappatura della cache.

4.4.1 Mappatura della Cache

Supponiamo che la memoria RAM e la cache siano divisi in **blocchi** di uguale dimensione. Quando andremo a spostare un blocco in cache, non sposteremo dunque il singolo bit, bensì tutto il singolo blocco.



In cache oltre ai blocchi in sé per sé troviamo:

- **bit valid:** che è un bit che ci indica se quel dato è ancora valido o se non è più presente nella RAM e, quindi, può essere anche eliminato nella cache se necessario.
- **bit dirty:** è un bit che ci indica che il dato presente in cache è stato modificato rispetto al dato presente in RAM e che, quindi, è necessaria una scrittura quando possibile. Possono esserci due strategie riguardo a questo bit, la prima è che si scrive sempre sulla RAM una volta che il dato è stato modificato la seconda è che si posticipa la scrittura in RAM fino a quando si può. La seconda opzione è consigliata quando si hanno tanti cambiamenti di dati così da non dover fare molte scritture.

Tornando alla mappatura da memoria RAM a cache troviamo le seguenti strategie:

- **Completamente associativa:** I dati possono andare in qualsiasi blocco libero. È efficiente per evitare conflitti in scrittura, ma lenta in lettura perché richiede la ricerca in tutta la cache.
- **Mappatura diretta:** Un indirizzo RAM può risiedere in un solo specifico blocco. Estremamente veloce da trovare, ma massimizza i conflitti se più dati utili competono per lo stesso blocco.
- **Set-associativa a N-vie:** È il compromesso tipico delle architetture moderne (es. associativa a 8 vie). I dati possono essere inseriti in uno specifico sottoinsieme (set) di blocchi.

Per scrivere codice efficiente bisogna gestire le tre C e le tre R:

- **Compulsory miss** → Inevitabili al primo accesso.
- **Capacity miss** (Spazio insufficiente) → Si contrasta con la regola **Ridurre**: usare tipi di dati più piccoli e lavorare su working set limitati.
- **Conflict miss** (Sovrascrittura di linee cache utili) → Si contrasta con le regole **Riorganizzare** (layout di memoria per migliorare la località) e **Riutilizzare** (completare tutte le operazioni su un dato mentre è ancora residente in cache).

4.4.2 Modello di accesso alla Memoria

Le prestazioni della memoria formano una superficie tridimensionale nota come *Memory Mountain*, che dipende strettamente da due fattori: la **dimensione del *working set*** (quanti dati stiamo manipolando) e lo **Stride** (il "passo" con cui leggiamo gli elementi in memoria).

Lavorare con un set di dati piccolo e uno stride unitario (elementi contigui) massimizza la banda passante (*Bandwidth*). Al contrario, scorrere grandi array saltando posizioni di memoria (stride elevato) distrugge le prestazioni perché spreca le linee di cache caricate.

Esempio: Impatto dello Stride nell'Accesso alla Memoria

La CPU non carica in cache singoli elementi interi, ma intere linee da 64 byte. Nel primo ciclo (`stride = 1`), un singolo accesso alla RAM carica in cache un blocco che contiene anche gli elementi successivi, rendendo le iterazioni seguenti istantanee.

```
#define N 1000000
int a[N];
int sum = 0;

/* Accesso sequenziale (Stride = 1) */
/* Veloce: sfrutta appieno l'intera cache line*/
for (int i = 0; i < N; i++) {
    sum += a[i];
}
```

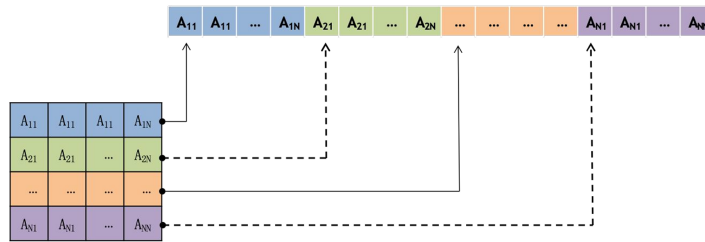
Se incrementiamo l'indice in modo non sequenziale, stiamo ignorando i dati portati in cache. Ad ogni salto, la CPU va in *cache miss* ed è costretta a pagare la penalità di centinaia di cicli per interrogare nuovamente la RAM.

```

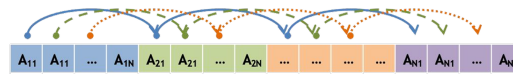
/* Accesso sparso (Stride grande) */
/* Lento: spreca quasi tutta la cache line e genero miss*/
for (int i = 0; i < N; i += 16) {
    sum += a[i];
}

```

Questo principio diventa critico quando si opera su **matrici multidimensionali**. In linguaggi come C e C++, le matrici non sono tabelle magiche, ma blocchi unidimensionali e continui di memoria. La convenzione di questi linguaggi è il **row-major order** (ordine per righe): gli elementi della stessa riga sono memorizzati consecutivamente in RAM. Al contrario, leggere una matrice scorrendola per colonne significa effettuare salti enormi (*stride*) nella memoria fisica, abbattendo radicalmente le prestazioni del programma (uno *stride* pari a $N \times S$ byte, dove N è la dimensione della riga e S la dimensione del dato).



La lettura della matrice quindi per colonne è chiaramente estremamente costosa dato che è necessario effettuare grandi salti andando quindi a passare da un punto della memoria a un altro anche provocando molti cache miss.



In genere, l'indice interno è quello che varia più velocemente e traccia la contiguità della memoria e, quindi, abbiamo che $M[i][j][k]$ è contiguo a $M[i][j][k+1]$. Considerando tutto quello che abbiamo detto, analizziamo questa sezione di codice:

Esempio: Lettura vs Scrittura con Stride

Nel primo caso leggiamo in modo contiguo ma scriviamo a salti. Nel secondo caso scriviamo in modo contiguo ma leggiamo a salti.

```
// Modalità 1: Lettura contigua, Scrittura con stride
for(int row = 0; row < N; row++) {
    for(int col = 0; col < N; col++) {
        A[col * N + row] = B[row * N + col];
    }
}

// Modalità 2: Scrittura contigua, Lettura con stride
for(int row = 0; row < N; row++) {
    for(int col = 0; col < N; col++) {
        A[row * N + col] = B[col * N + row];
    }
}
```

In entrambi i casi abbiamo uno stride, tuttavia, questi non sono uguali. Quando si modifica il contenuto di una variabile, tale modifica equivale a sovrascrivere una posizione di memoria. Quando la variabile viene caricata nella memoria cache, ciò che viene caricata è una copia della variabile, che mantiene la sua posizione originale nella memoria principale. Quindi, se modifichiamo il suo valore, dobbiamo modificarlo solo nella cache o anche nella memoria principale?

- **Write-Through:** La modifica viene scritta immediatamente sia in cache che in RAM mantenendoli coerenti.
- **Write-Back:** La modifica viene apportata solo in cache e la RAM viene aggiornata solo quando la linea di cache viene sostituita.

In caso di *write-miss* in una cache di tipo write-through possiamo allocare prima il blocco in cache e poi effettuare la scrittura in entrambi (write-allocate) o scrivere direttamente sulla memoria principale (non-write-allocate). Nell'ultimo caso la scrittura a raffica è conveniente per evitare continue allocazioni in cache.

4.4.3 Organizzazione dei Dati

Poiché lavoriamo con cache che hanno generalmente dimensioni molto piccole, organizzare i dati per far sì che questi stiano dentro la cache diventa fondamentale per sfruttare il principio di località spaziale.

Scansione per blocchi

Oltre alla classica scansione per righe/colonne esiste un altro tipo di scansione detta **scansione per blocchi** fatta appositamente per essere sfruttata quando lavoriamo con operazioni molto pesanti come moltiplicazioni o inversioni di matrice. L'idea è che invece di attraversare intere righe o colonne, dividiamo la matrice in **sottomatrici più piccole** dette blocchi scegliendo la dimensione facendo sì che i blocchi delle matrici coinvolte siano tutti in cache così da abbattere i cache miss.

```

0-1-2-3 16-17-18-19
4-5-6-7 20-21-22-23
8-9-10-11 24-25-26-27
12-13-14-15 28-29-30-31
32-33-34-35 48-49-50-51
36-37-38-39 52-53-54-55
40-41-42-43 56-57-58-59
44-45-46-47 60-61-62-63

```

Anziché codificare manualmente la dimensione del blocco, è possibile memorizzare direttamente la matrice in RAM utilizzando pattern non lineari, come la **Curva di Morton** (o Z-order). Questo ordinamento mappa dati multidimensionali in 1D mantenendo fisicamente vicini in memoria i punti che sono vicini nello spazio 2D o 3D, migliorando le prestazioni medie su architetture diverse senza bisogno di un *tuning* manuale dei blocchi.

Campi Caldi e Campi Freddi

Un errore comune nell'ottimizzazione è progettare strutture dati (`struct`) che mescolano dati ad accesso frequente ("caldi") con dati raramente utilizzati ("freddi"). Nelle liste concatenate, ad esempio, i puntatori e le chiavi di ricerca vengono letti ad ogni iterazione, mentre il carico utile (payload) viene letto solo in caso di successo.

Esempio: Ottimizzazione delle Strutture Dati

Nella prima versione, l'inserimento di un grosso array tra la chiave e il puntatore al nodo successivo allontana fisicamente i due dati. Questo distrugge la località spaziale: la cache verrà inondata da byte inutili caricati per via del payload (`my_data`).

```
// Versione NON ottimizzata (Campi mischiati)
struct my_node {
    double key;                // HOT
    char my_data[300];         // COLD: Allontana key e next
    my_node *next_node;        // HOT
};
```

La soluzione consiste nell'estrarre i dati "freddi" in una struttura separata allocata altrove, lasciando nella struttura principale solo un puntatore. In questo modo, `key` e `next_node` diventano adiacenti in memoria e verranno caricati insieme in una singola e veloce linea di cache.

```
// Versione OTTIMIZZATA (Separazione Hot/Cold)
struct my_data {
    char data[300];
};

struct my_node {
    double key;                // HOT
    my_node *next_node;        // HOT: Ora è adiacente alla chiave!
    struct my_data *payload;    // COLD: Puntatore ai dati esterni
};
```

4.5 Branch condizionali

Ogni volta che la sequenza di operazioni da eseguire o i dati da elaborare dipendono da una condizione (ad esempio, l'esito di un test 'if'), il programma effettua un'esecuzione condizionale. Le architetture moderne gestiscono questa situazione a basso livello in due modi distinti: modificando il flusso di controllo oppure modificando il flusso di dati.

Flusso di Controllo

A livello macchina, il controllo viene passato a una sezione diversa del codice tramite istruzioni di salto (*jump*). Queste possono essere:

- **Incondizionate** (`jmp`): Il salto avviene sempre.
- **Condizionali** (`je`, `jne`, `jg`, `jl`, ecc.): L'esecuzione dipende dall'esito di un test precedente. Queste istruzioni leggono i bit del **registro dei flag** (es. *Zero Flag*, *Sign Flag*, *Carry Flag*), che memorizza lo stato dell'ultima operazione logico-aritmetica.

In questo scenario, il codice generato dal compilatore è pieno di bivi e diramazioni fisiche.

Flusso di Dati

Le CPU moderne offrono un meccanismo alternativo: invece di saltare a blocchi di codice diversi, calcolano in anticipo i possibili risultati e poi spostano nei registri finali solo il dato corretto (es. Tramite l'istruzione assembly `cmov - conditional move`). Questo approccio non genera bivi nel codice, mantenendo un flusso di esecuzione lineare e altamente prevedibile. Tuttavia, è applicabile solo a scenari semplici poiché altrimenti il numero di operazioni necessarie al calcolo supera il vantaggio dato dal calcolare tutti i risultati.

4.5.1 Branch Predictor

Per mantenere le pipeline di esecuzione sempre piene, le CPU utilizzano un componente hardware chiamato **Branch Predictor**, che cerca di indovinare in anticipo l'esito di una diramazione prima che questa venga effettivamente valutata per anticiparne i calcoli.

I migliori predittori moderni hanno un'accuratezza del 95%. Tuttavia, quando il predittore sbaglia (**Branch Miss** o *Misprediction*), la CPU è costretta a svuotare l'intera pipeline scartando il lavoro fatto in anticipo. Questa penalità costa tipicamente dai 10 ai 30 cicli di clock per ogni errore.

L'impatto statistico è enorme: supponiamo di avere un blocco di 140 istruzioni in cui 1 istruzione su 7 è un branch (quindi 20 branch in totale). Assumendo eventi indipendenti, la probabilità che la pipeline *non* venga mai svuotata con un predittore accurato al 95% crolla drasticamente:

$$P(\text{noflush}) = 0.95^{20} \approx 0.3585 \Rightarrow 35.85\%$$

Con un predittore al 90%, la probabilità scende ad appena il 12.16%. Per questo motivo, i branch condizionali all'interno dei cicli (*loop*) sono i nemici principali delle prestazioni e vanno evitati o ristrutturati.

Ecco perché la seconda strategia che abbiamo visto, la modifica condizionale del flusso di dati, è preferibile quando possibile e il compilatore cercherà di utilizzarla il più possibile poiché non genera istruzioni di salto e il flusso di esecuzione è lineare e perfettamente prevedibile. Tuttavia, come abbiamo detto, si applica solo a un limitato sottoinsieme di casi.

Una cosa da tenere in considerazione che provoca un miglioramento delle prestazioni è che il branch con la condizione vera è quello che sta più vicina rispetto all'`if` stesso. Dunque, per evitare salti da una parte all'altra del codice, sarebbe bene che il branch vero sia quello che è più probabile che venga utilizzato.

Idealmente vorremmo evitare i branch condizionali il più possibile all'interno dei cicli:

- Spostandoli fuori da Loop e scrivendo Loop più specializzati.
- Esecuzioni di variabili/quantità impostate fuori dal ciclo.
- Utilizzando puntatori alle funzioni invece di selezionare funzioni all'interno di un ciclo.
- Uso di operazioni diverse per evitare branch.

4.5.2 Tecniche per limitare i Branch

In genere ci sono due casi possibili:

- Le variabili testate nelle condizioni non cambiano durante il ciclo. Questo è il caso semplice e con poca ristrutturazione si hanno grandi miglioramenti.
- Le variabili cambiano nel ciclo. Qui abbiamo bisogno di molto lavoro in più per cercare di risolvere il problema.

Vediamo adesso una carrellata di esempi.

Sollevamento del loop

Se una condizione testata all'interno di un ciclo non dipende dall'iteratore, è un grave errore ripeterla ad ogni passo. La diramazione va spostata *fuori* dal ciclo, scrivendo versioni specializzate del loop stesso.

```
// Versione NON ottimizzata: il test viene ripetuto 'top' volte
for(int i = 1; i < top; i++) {
    if (value > 1) do_something();
    else          do_something_else();
}

// Versione OTTIMIZZATA: il test viene eseguito UNA sola volta
if (value > 1) {
    for (int i = 1; i < top; i++) do_something();
} else {
    for (int i = 1; i < top; i++) do_something_else();
}
```

Nota: Nei casi di scelte multiple basate su uno stato costante, è nettamente preferibile usare un costrutto `switch` invece di una cascata di `if/else`. Il compilatore traduce lo `switch` in una tabella statica di puntatori detta **jump table** a cui si accede in tempo costante $O(1)$ senza diramazioni condizionali.

Flussi di dati imprevedibili

Quando la condizione dipende dai dati stessi (es. l'elemento corrente di un array casuale), il predittore fallirà continuamente. Spesso si può sostituire l'if con un'operazione booleana moltiplicativa.

```
// Versione NON ottimizzata: Rischio altissimo di Branch Miss
for (int i = 0; i < SIZE; i++) {
    if (data[i] > PIVOT)
        sum += data[i];
}

// Versione OTTIMIZZATA: Nessun salto! Il flusso è lineare.
for (int i = 0; i < SIZE; i++) {
    sum += (data[i] > PIVOT) * data[i];
}
```

Poiché ($data[i] > PIVOT$) viene valutato come 1 (Vero) o 0 (Falso), è possibile sfruttare questa logica per eliminare il branch.

Ordinamento degli elementi

Un caso classico è lo scambio condizionale di valori tra due array per mantenere un ordinamento. L'uso esplicito dell'if distrugge le prestazioni. È preferibile usare operatori condizionali ternari o maschere bit a bit che il compilatore può tradurre in istruzioni `cmov` (flusso di dati condizionale senza salti).

```
// Versione NON ottimizzata
for (int i = 0; i < SIZE; i++) {
    if (A[i] < B[i]) {
        int t = B[i];
        B[i] = A[i];
        A[i] = t;
    }
}

// Versione OTTIMIZZATA: Sfrutta min e max
for (int i = 0; i < SIZE; i++) {
    int min = (A[i] > B[i]) ? B[i] : A[i];
    int max = (A[i] >= B[i]) ? A[i] : B[i];
}
```

```
A[i] = max;  
B[i] = min;  
}
```

Cambiare il POV

A volte il modo migliore per rimuovere i branch è ripensare i limiti stessi dei cicli, invece di usare ‘if’ interni per discriminare le aree spaziali (es. in una matrice).

Vogliamo riempire la diagonale di una matrice $M \times N$ con 0.0, il triangolo superiore con 1.0 e quello inferiore con -1.0 .

```
// Versione NON ottimizzata: if/else dentro i loop annidati  
for (int j = 0; j < N; j++) {  
    for (int i = 0; i < M; i++) {  
        if (i > j)          matrix[j][i] = 1.0;  
        else if (i <= j) matrix[j][i] = -1.0;  
        else                matrix[j][i] = 0.0;  
    }  
}  
  
// Versione OTTIMIZZATA: Loop separati basati sulla geometria  
for (int j = 0; j < N; j++) {  
    int i;  
    for (i = 0; i < j; i++)          // Fino alla diagonale  
        matrix[j][i] = -1.0;  
  
    matrix[j][i] = 0.0;              // Sulla diagonale esatta  
  
    for (i = j + 1; i < M; i++)      // Dopo la diagonale  
        matrix[j][i] = 1.0;  
}
```

4.6 Ottimizzazione dei Loop

Ottimizzare i cicli significa massimizzare il riutilizzo dei dati e mantenere le pipeline della CPU sempre piene. Per capire quanto un ciclo sia ottimizzabile, si introduce il concetto di **Intensità Aritmetica** (A_I), definita come

il rapporto tra il numero di operazioni eseguite $f(n)$ e la quantità di dati trasferiti n :

$$A_I = \frac{f(n)}{n} \quad (4.2)$$

In base all'intensità aritmetica, i cicli si classificano in tre categorie:

1. $O(N)/O(N)$: Es. Addizioni vettoriali o prodotti scalari. Il potenziale di ottimizzazione è limitato perché il limite è imposto dalla larghezza di banda della memoria. Le migliori derivano dall'evitare accessi ripetuti (es. *Loop Fusion*).
2. $O(N^2)/O(N^2)$: Es. Moltiplicazione matrice-vettore. Offre maggiori opportunità di ottimizzazione sfruttando la località spaziale e temporale.
3. $O(N^3)/O(N^2)$: Es. Moltiplicazione matrice-matrice. Ha un enorme potenziale di ottimizzazione perché i dati caricati in cache possono essere riutilizzati molteplici volte per fare calcoli.

4.6.1 Ottimizzazioni per Cicli a Bassa Intensità: Loop Fusion

Quando si elaborano cicli $O(N)/O(N)$, l'obiettivo principale è ridurre il traffico verso la memoria principale. Se due cicli consecutivi scorrono gli stessi array, fonderli in un unico ciclo (**Loop Fusion**) dimezza i caricamenti dalla RAM.

Esempio: Fusione dei Loop (Loop Fusion)

```
//B[i] viene caricato dalla RAM due volte
for(int i=0; i<N; i++)
    A[i] = B[i] * C[i];
for(int i=0; i<N; i++)
    Q[i] = B[i] + D[i];

//OTTIMIZZATA: Fusione, B[i] è letto una sola volta.
for(int i=0; i<N; i++) {
    A[i] = B[i] * C[i];
    Q[i] = B[i] + D[i];
}
```

4.6.2 Ottimizzazioni per Cicli $O(N^2)$: Unroll & Jam

Nei cicli annidati, è possibile ridurre l'overhead e migliorare il riutilizzo dei dati srotolando il ciclo esterno e fondendolo con quello interno (**Unroll & Jam**).

Tuttavia, bisogna prestare attenzione al **Register Spilling**: se si srotola troppo (es. un fattore m troppo grande), la CPU esaurisce i registri hardware disponibili ed è costretta a riversare continuamente i dati temporanei nella cache, rallentando drasticamente il calcolo (*codice gonfiato*).

Esempio: Unroll & Jam in una Moltiplicazione Matrice-Vettore

```
// 1. Versione Base (3*N^2 accessi)
for(int i=0; i<N; i++)
    for(int j=0; j<N; j++)
        C[i] += A[i][j] * B[j];

// 2. Non accediamo a C[i] ma a una variabile locale
for(int i=0; i<N; i++) {
    double c_temp = C[i];
    for(int j=0; j<N; j++)
        c_temp += A[i][j] * B[j];
    C[i] = c_temp;
}

// 3. Unroll & Jam
for(int i=0; i<N; i+=2) {
    for(int j=0; j<N; j++) {
        double b_temp = B[j];
        C[i]   += A[i][j]   * b_temp;
        C[i+1] += A[i+1][j] * b_temp;
    }
}
```

4.6.3 Srotolamento del Ciclo (Loop Unrolling) e Aritmetica Floating Point

Lo srotolamento (*unrolling*) è una trasformazione fondamentale che espone il parallelismo a livello di istruzione (ILP) e riduce il costo di aggiornamento del contatore.

Esiste però un'enorme differenza tra variabili intere e in virgola mobile. Come discusso in precedenza, **la matematica in virgola mobile non è associativa**. Se si compila una semplice somma di array ('S += A[i]') usando il

tipo `int`, il compilatore vettorizza automaticamente il codice massimizzando le prestazioni. Se si usa il tipo `double`, il compilatore **non** ottimizzerà il ciclo, perché alterare l'ordine delle somme cambierebbe il risultato numerico a causa degli arrotondamenti. È quindi responsabilità del programmatore riassociare esplicitamente le operazioni.

Esempio: Srotolamento e Riassociazione Esplicita per i Double

Per sfruttare le porte vettoriali della CPU con i 'double', dobbiamo srotolare manualmente il ciclo e forzare un nuovo ordine di valutazione tramite le parentesi, esponendo operazioni indipendenti.

```
// la CPU deve aspettare la fine della somma precedente
for (int i=0; i<N; i++)
    S = S + A[i];

// Versione Srotolata 2x1
// (Riduce l'overhead del ciclo, ma resta sequenziale)
for (int i=0; i<N-2; i+=2)
    S = (S + A[i]) + A[i+1];

// Versione Srotolata e RIASSOCIATA (Parallelismo esposto!)
for (int i=0; i<N-2; i+=2)
    S = S + (A[i] + A[i+1]);
```

Nota: Modificando l'ordine in '(A[i] + A[i+1])', le due somme elementari sono indipendenti dal valore precedente di 'S'. La CPU può calcolare questo blocco in parallelo nelle proprie pipeline vettoriali (es. istruzioni AVX `vaddpd`) e sommarlo ad 'S' solo alla fine, abbattendo drasticamente il tempo di esecuzione. Ricordarsi sempre di gestire la "coda" (tail) del ciclo per gli elementi residui non multipli del fattore di srotolamento.

4.6.4 Esempio Pratico $O(N^3)/O(N^2)$: Moltiplicazione Matrice-Matrice (MMM)

La moltiplicazione di due matrici quadrate $C = A \times B$ richiede $2N^3$ operazioni floating point (flop). A causa dell'ordine *row-major* del linguaggio C, attraversare la matrice B per colonne provoca un'enorme quantità di *cache miss*. Un'implementazione ingenua genera quasi un cache miss per ogni flop, degradando le prestazioni.

Esistono tre fasi progressive per ottimizzare questo problema computazionale:

1. **Trasposizione preventiva:** Trasporre la matrice B in memoria prima dei calcoli permette di accedere ad entrambe le matrici sequenzialmente (per righe).
2. **Scambio dei cicli (Loop Swapping):** Cambiare l'ordine di annidamento dai classici $i-j-k$ a $i-k-j$. Questo approccio accede agli elementi di C in modo contiguo e legge B in ordine di riga, riducendo le letture dalla RAM.
3. **Elaborazione a Blocchi (Tiling):** Mantenere in cache sottomatrici fisse (blocchi o *tiles*). Invece di calcolare un'intera riga per volta, si accumulano risultati parziali su un blocco ristretto di C sfruttando mini-porzioni di A e B caricate in L1.

Impatto sulle metriche: I profiler hardware dimostrano che passando dall'implementazione ingenua (`O0 naive`) all'implementazione a blocchi (`O3 tailed`), i miss della cache L1 crollano vicino allo zero, l'IPC (Instructions Per Cycle) si avvicina al massimo teorico dell'architettura e il tempo di esecuzione scala perfettamente senza subire crolli all'aumentare di N .

Capitolo 5

Parallelismo e Scalabilità

Definiamo il parallelismo come l'**esecuzione simultanea** da parte di più unità di lavoro al fine di **ridurre il tempo di esecuzione o aumentare la dimensione del problema risolvibile**. Facciamo una distinzione tra esecuzione in **contemporanea (concurrency)** ed esecuzione in **parallelo**:

- **Concurrency**: La concurrency è la capacità di un sistema di gestire più compiti (task) nello stesso intervallo di tempo, ma **non necessariamente nello stesso istante**.
- **Parallelism**: Il parallelismo si verifica quando più compiti vengono eseguiti fisicamente nello stesso istante.

Adesso che abbiamo parlato dell'ottimizzazione, ci rendiamo facilmente conto di quanto il parallelismo sia ormai fondamentale nelle architetture odierne. Senza questo strumento sarebbe impossibile risolvere problemi complessi a cui lavoriamo oggi e, nonostante questo strumento, siamo ancora molto lontani dall'avere le prestazioni che vorremmo.

5.1 Modelli di parallelismo

Parliamo dunque nel dettaglio di **modelli di parallelismo**. Questi definiscono:

- Come il lavoro viene suddiviso;
- Come le unità di elaborazione interagiscono tra loro;
- Il modello di memoria da adottare;
- I costi di **comunicazione, sincronizzazione e gestione dei dati**.

Il modello ha, dunque, un'influenza riguardo al come dobbiamo progettare il nostro codice e di come questo possa scalare. Possiamo dividere il parallelismo sulla base di **come si suddivide il lavoro**:

- **Data Parallelism**: La stessa operazione è applicata a porzioni o **chunk** di dati diversi. Questo tipo di parallelismo è dominante in HPC scientifico.
- **Task Parallelism**: Compiti diversi vengono eseguiti in parallelo. Tipico in applicazioni eterogenee.

O sulla base di come è organizzata la memoria:

- **Shared memory**: Secondo questa organizzazione abbiamo uno **spazio di indirizzamento unico**. Abbiamo quindi che i thread/processi guardano tutti la stessa porzione di memoria e, dunque, comunicano **implicitamente** proprio guardando la memoria. Questo modello di memoria è generalmente adottato sui singoli nodi poiché non è possibile collegare più di un certo numero di nodi alla stessa memoria per problemi di sovraffollamento e controlli di coerenza tra di essi.
- **Distributed memory**: Ogni processo ha una memoria **privata** che, quindi, nel caso in cui avesse bisogno di comunicare con un altro processo, dovrà farlo mediante messaggi **espliciti**. Questo modello è generalmente usato nei cluster così che ogni nodo del cluster abbia la sua memoria dato che questi sono indipendenti gli uni dagli altri, ciò è molto più scalabile.
- **Ibrido**: I sistemi HPC moderni sono spesso **ibridi** per cercare di prendere il meglio dall'uno e dall'altro.

Modelli a Memoria Condivisa

Come già detto, nel modello a memoria condivisa, tutte le unità di esecuzione condividono lo spazio degli indirizzi. Le variabili sono dunque visibili da tutte le unità di esecuzione e la comunicazione avviene implicitamente tramite l'accesso a memoria condivisa.

Questo modello ha il vantaggio di avere una comunicazione intra-nodo estremamente rapida al costo, però, di dover **gestire i meccanismi di sincronizzazione** dovuti ai problemi di race condition ecc.

Modelli a Memoria Distribuita

Nei modelli a memoria distribuita ogni processo ha la sua memoria privata e i dati che si scambiano i vari processi devono essere scambiati esplicitamente mediante messaggi o scambi di dati. Questo garantisce un **maggiore controllo sulla distribuzione dei dati** a patto di avere una maggiore complessità riguardo alla programmazione degli stessi.

Questo modello è **altamente scalabile** e adatto a **sistemi multi-nodo**.

5.2 Scalabilità

La scalabilità è la capacità di un sistema parallelo di migliorare le prestazioni al crescere delle risorse utilizzate.

Distinguiamo tra **Strong Scaling** e **Weak Scaling**:

- **Strong Scaling**: Questo scenario suppone di avere un problema di dimensione fissa e che, all'aumentare del numero di processi di lavoro, si misura una riduzione del tempo di risoluzione del problema.
- **Weak Scaling**: Qui, invece, abbiamo uno scenario con un problema che cresce all'aumentare dei processi. In questo caso, il nostro obiettivo sarà quello non di andare a diminuire il tempo di risoluzione del problema, bensì quello di farlo rimanere costante all'aumentare della dimensione del problema.

Sia $T(1)$ il tempo di risoluzione con un processo e $T(p)$ il tempo di risoluzione con p processi, definiamo:

- **Speedup**: Come il rapporto tra il tempo di esecuzione con un singolo processo e il tempo con p processi, cioè:

$$S(p) = \frac{T(1)}{T(p)}$$

Questo va a misurare quanto il nostro problema migliora in termini di tempo all'aumentare dei processi che lo elaborano. Idealmente, vorremmo che $S(p) = p$.

- **Efficienza**: Questa è definita come il rapporto tra lo Speedup e p , cioè:

$$E(p) = \frac{S(p)}{p}$$

Questo va a misurare quanto il nostro problema scala bene all'aumentare dei processi che lo elaborano. Idealmente, vorremmo che $E(p) = 1$.

In realtà, per quanto vorremmo che lo Speedup sia lineare, non è così poiché:

- **Parte Seriale:** Una parte del programma potrebbe non essere parallelizzabile e, dunque, l'aumentare i processi non è una soluzione.
- **Comunicazione:** Inevitabilmente, a causa dell'uso di molti processi, questi hanno la necessità di comunicare e, ciò, introduce una latenza.
- **Sincronizzazione:** Poiché lavoriamo con sezioni critiche ecc. abbiamo la necessità di introdurre delle attese.
- **Overhead di gestione:** Dovuto allo scheduling o alla creazione dei thread/processi.
- **Load imbalance:** Cioè alcuni processi terminano prima di altri.

Legge di Amdahl

Sia α la frazione parallelizzabile e $1 - \alpha$ la parte seriale. Il tempo di esecuzione con p processi è:

$$T(p) = T(1) * [(1 - \alpha) + \frac{\alpha}{p}]$$

Lo Speedup è, invece:

$$S(p) = \frac{1}{(1 - \alpha) + \frac{\alpha}{p}}$$

Il limite teorico per p che tende a infinito è:

$$\lim_{p \rightarrow \infty} S(p) = \frac{1}{1 - \alpha}$$

Questo dipende dal fatto che, anche con infiniti processi, la parte seriale rimane un limite non superabile.

Legge di Gustafson

La legge di Gustafson è applicabile nel caso di **weak scaling** in cui il problema cresce con il numero di processi ma la parte seriale rimane costante. Quella che, dunque, non sarà costante, sarà proprio la parte parallelizzabile che crescerà con p . Lo Speedup diventerà:

$$S(p) = p - (1 - \alpha)(p - 1)$$

Se α fosse vicino a 1, cioè la parte parallelizzabile fosse quasi la totalità del programma, allora lo Speedup crescerebbe quasi linearmente. Per questa legge non abbiamo un limite come nel caso della legge di Amdahl.

Abbiamo un vantaggio nell'aggiungere processi quando:

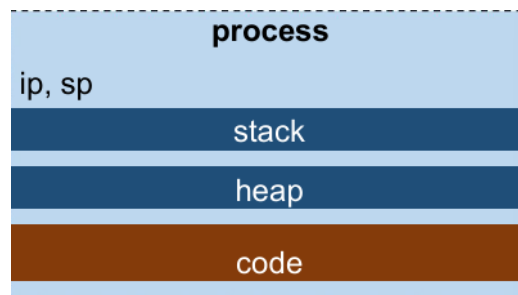
- La parte seriale è minima.
- Il problema è sufficientemente grande a tal punto che il calcolo domina la comunicazione.
- Il carico è ben bilanciato.

Capitolo 6

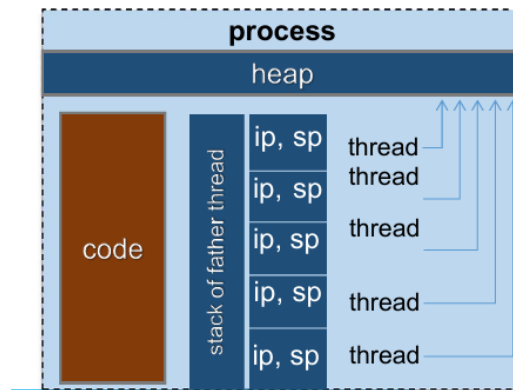
OpenMP

Definiamo un **processo** come una sequenza indipendente di istruzioni e l'insieme di risorse necessarie alla sua esecuzione. Questi, infatti, devono accedere a uno spazio di memoria protetto e a delle risorse del sistema.

Possono esistere più processi in esecuzione che eseguono lo stesso programma in maniera del tutto indipendente che possiedono, quindi, diverse risorse del sistema indipendenti li uni dalle altre.



Al contrario, un **thread** è un'istanza indipendente di esecuzione di un codice all'**interno di un processo**. Possono esserci, dunque, più thread in esecuzione sullo stesso processo. Ognuno di questi, condivide lo stesso codice, lo stesso spazio di indirizzamento e le stesse risorse del processo padre. Siamo in un modello a **memoria condivisa** in cui ogni unità di elaborazione (i thread) condivide la stessa memoria ma con un piccolo spazio **privato**, come in un'architettura NUMA.



In generale, generare thread all'interno di un processo è molto meno costoso che creare processi.

A seconda dell'architettura hardware, si utilizzano paradigmi software differenti.

- **Shared-Memory (es. OpenMP):** Un singolo processo univoco genera un certo numero di thread. Poiché esiste uno spazio di memoria univoco (l'Heap del processo), la comunicazione tra thread avviene in modo *implicito* leggendo e scrivendo le stesse variabili in memoria.
- **Distributed-Memory (es. MPI):** Vengono creati N processi distinti, ognuno con la propria copia del codice e il proprio spazio di memoria privato e isolato. Un processo non può fisicamente accedere alla memoria di un altro. La comunicazione deve avvenire obbligatoriamente in modo *esplicito* tramite lo scambio di messaggi sulla rete.

6.1 Cosa è OpenMP

OpenMP (Open specifications for MultiProcessing) è un'API standard per la programmazione parallela a memoria condivisa. Consente di scrivere programmi multithread con un comportamento standard attraverso l'utilizzo di un set di direttive del compilatore da inserire nel codice sorgente. OpenMP permette di parallelizzare il codice sia suddividendo automaticamente piccoli lavori (come le iterazioni di un ciclo), sia assegnando esplicitamente compiti più grandi ai singoli thread.

OpenMP è composto da 3 componenti:

1. **Direttive del Compilatore:** In C/C++ assumono la forma di `#pragma` e forniscono indicazioni su come gestire i thread.

2. **Librerie di Runtime:** Funzioni richiamabili dal codice (es. per sapere quanti thread stanno lavorando).
3. **Variabili d'Ambiente:** Parametri impostati dall'utente a livello di sistema operativo (es. per definire quanti thread generare o per gestire l'affinità core-memoria).

I vantaggi di un approccio basato sulle direttive sono:

- **Astrazione:** Le sottigliezze di pthread e gli aspetti specifici dell'hardware sono nascosti. È possibile concentrarsi sui dati e sul flusso di lavoro molto più facilmente.
- **Efficienza:** La curva di apprendimento per ottenere risultati ragionevoli è molto più semplice. La progettazione del codice è più semplice, il rapporto risultato/sforzo è favorevole rispetto a pthread.
- **Approccio incrementale:** Non è necessario riscrivere l'intero codice. Si inizia concentrandosi solo su alcune sezioni, seguendo i suggerimenti del profiling.
- **Portabilità:** Il compilatore si occuperà di questo per noi. È comunque necessario sviluppare una progettazione in grado di adattarsi a diverse topologie.
- **Un'unica fonte:** Attraverso la compilazione condizionale, le versioni seriali e parallele possono coesistere facilmente.

6.2 Modello Fork-Join

OpenMP basa il suo funzionamento sul modello **Fork-Join**. Il programma inizia l'esecuzione come un normale programma sequenziale guidato da un **singolo thread master**.

- **Fork:** Quando il thread master incontra una direttiva specifica (una regione parallela), genera un *team di thread* figli per eseguire il lavoro in parallelo. Il master sospende la sua attività singola e si unisce al pool lavorando come "thread 0".
- **Join:** Al termine della regione parallela, i thread si sincronizzano. I thread figli terminano (o si mettono in pausa) e il solo thread master riprende l'esecuzione seriale del codice.

Il modello offre flessibilità nella gestione delle risorse, ma presenta vincoli stringenti riguardo alla struttura del codice:

- **Dipendenze nei Loop:** Le dipendenze "trasportate" dai cicli (quando un'iterazione ha bisogno del risultato della precedente) rappresentano il principale ostacolo all'accelerazione parallela.
- **Parallelismo Annidato (Nested):** È esplicitamente consentito creare regioni parallele all'interno di altre regioni parallele.
- **Modifica Dinamica:** Il numero di thread non è necessariamente fisso ma può essere modificato dinamicamente prima di entrare in una nuova regione parallela.

6.2.1 Direttive

Le direttive OpenMP si applicano a **blocchi di codice strutturati**, ovvero blocchi che hanno un singolo punto di ingresso e un singolo punto di uscita, senza istruzioni di salto (branch) incontrollati al loro interno. La direttiva principale per avviare il parallelismo è:

```
#pragma omp parallel{  
    // Codice eseguito in parallelo da tutti i thread del team  
}
```

Queste direttive permettono di:

- Creare team di Thread per eseguire in parallelo.
- Gestire il carico di lavoro tra i thread.
- Specificare quali sono le parti di memoria condivisa e quali no.
- Gestire l'aggiornamento delle regioni condivise.
- Sincronizzare thread e determinare operazioni atomiche e/o esclusive.

Possiamo fare riferimento a due cose in particolare:

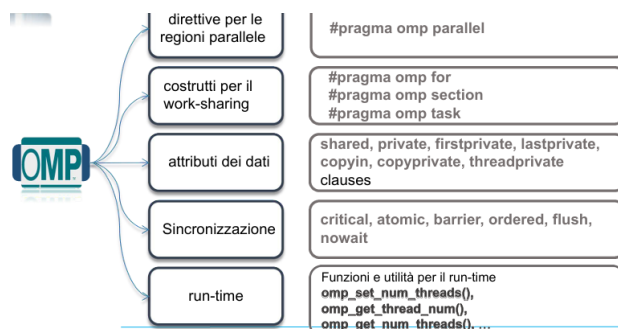
- **Static Extent:** È l'ambito lessicale del blocco strutturato. Riguarda solo le righe di codice scritte fisicamente e visivamente all'interno delle parentesi graffe subito dopo il `#pragma`.

- **Dynamic Extent:** Se all'interno della regione parallela viene chiamata una funzione esterna, anche il codice di quella funzione verrà eseguito in parallelo da tutti i thread. L'estensione dinamica include l'estensione statica originale *più* tutte le istruzioni e le ulteriori chiamate lungo l'albero delle chiamate.

Le funzioni chiamate nell'estensione dinamica possono contenere direttive OpenMP aggiuntive dette **direttive orfane**.

Uno dei grandi vantaggi di OpenMP è l'approccio incrementale. Non dobbiamo riscrivere tutto il codice, ma possiamo aggiungere gradualmente i pragma nei colli di bottiglia. Inoltre, lo stesso identico codice sorgente può essere compilato sia in modalità parallela che in modalità puramente sequenziale ignorando i pragma.

Quando al compilatore viene passata l'istruzione di attivare OpenMP (es. tramite il flag `-fopenmp` in GCC), esso definisce automaticamente una macro di sistema chiamata `_OPENMP`.



6.3 Regioni Parallele

Come abbiamo visto, OpenMP utilizza il modello fork-join. Il lavoro parallelo si svolge all'interno di **regioni parallele**, che in C/C++ vengono definite applicando una direttiva a un blocco di codice strutturato.

Quando il compilatore incontra la direttiva `#pragma omp parallel`, crea un pool di thread. Ognuno di essi riceve un ID univoco (da 0 a $N - 1$): **ogni thread riceve un proprio Stack privato e un proprio Instruction Pointer (IP)** separati da quelli del processo padre. Avranno tutti lo stesso **process id (PID)** ma diverso **Thread ID (TID)**. Le direttive in generale in C/C++ sono di questo tipo:

```
#pragma omp <directive> [<clause> [ <clause> [...]]]
```

Ci sono tanti tipi di costrutti omp, tra cui:

- `#pragma omp parallel _some_clauses_here_`: Una regione parallela può essere una linea singola.
- `#pragma omp parallel _some_clauses_here_ { ... }`: Nessun limite sulla dimensione del codice compreso tra `{...}`.
- `#pragma omp parallel for _some_clauses_here_ { ... }`: Un costruttore specifico per i cicli `for`.
- `#pragma omp sections _some_clauses_here_ { ... }`: Un costrutto più generale per il *work-sharing*.
- `#pragma omp task _some_clauses_here_ { ... }`: Questo permette un parallelismo basato sui task.

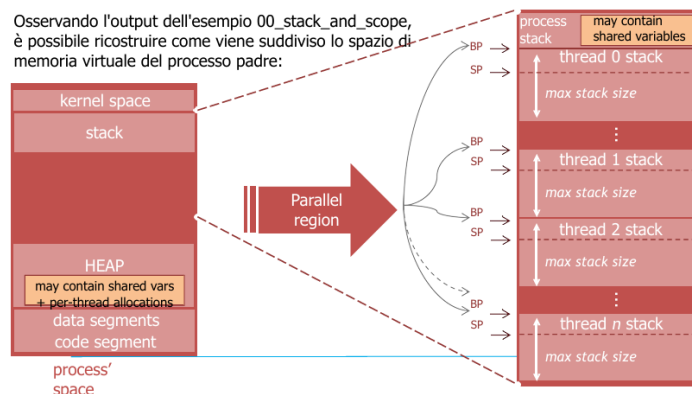
Questi li vedremo più avanti nel dettaglio.

La creazione dello stack per ogni thread non è "gratuita". L'hardware deve allocare fisicamente porzioni di memoria virtuale per ospitare i dati privati di ciascun thread. La dimensione predefinita di questo stack varia a seconda del compilatore (es. ~ 4 MB con GCC, fino a 132 MB con Clang).

Questo genera due scenari di rischio architetturale:

- **Out of Memory (OOM)**: Se generi troppi thread o se la dimensione dello stack è troppo grande, potresti esaurire la memoria fisica disponibile nel sistema.
- **Segmentation Fault**: Al contrario, se lo stack predefinito è troppo piccolo e il tuo thread tenta di allocare grossi array locali, sfonderà il limite dello stack causando un errore di segmentazione.

Soluzione operativa: L'utente può controllare esplicitamente questa dimensione esportando la variabile d'ambiente `OMP_STACKSIZE` (es. `export OMP_STACKSIZE=1M`) prima di eseguire il codice.



Di norma, inoltre, il numero di thread che possono essere generati da un processo è limitato dal sistema. Ad esempio, Linux imposta che il numero massimo di thread è la quantità di memoria fisica divisa per la quantità di memoria necessaria per descrivere ed eseguire un thread per un fattore che impedisce che la memoria sia tutta usata per rendere attivi i thread.

Entro questo limite, possiamo modificare il comportamento di OpenMP con `OMP_THREAD_LIMIT`.

6.3.1 Gestione della Regione Parallela

Innanzitutto, vediamo come si costruisce una classica regione parallela:

```
#pragma omp parallel
{
    int id = omp_get_thread_num();

    #pragma omp single
    {
        int nthreads = omp_get_num_threads();
    }

    #pragma omp barrier //forza tutti i thread ad arrivare allo stesso punto
    //il rischio è causare rallentamenti per attendere la fine di tutti i thread
    printf("Hello from thread!");
}
```

Possiamo ottenere diversi comportamenti con alcune aggiunte dopo la parola chiave *parallel*:

- **if(condizione)**: questo if permette di aprire la regione parallela solo se la condizione è vera. Utile per evitare l'overhead dovuto alla creazione di troppi thread anche nei casi di in cui vi siano pochi dati. La sezione di codice viene seguita a prescindere ma, nel caso in cui la condizione sia falsa, allora non verrebbero creati thread.
- **num_threads(n)**: questa condizione forza il processo a creare esattamente n thread ignorando le eventuali variabili di ambiente impostate.
- **proc.bind**: Serve a dare direttive sul dove far lavorare il thread in termini di core fisici. Senza questa direttiva il sistema operativo potrebbe decidere di spostarli come più gli aggrada, creando un overhead causato dallo spreco di cache.

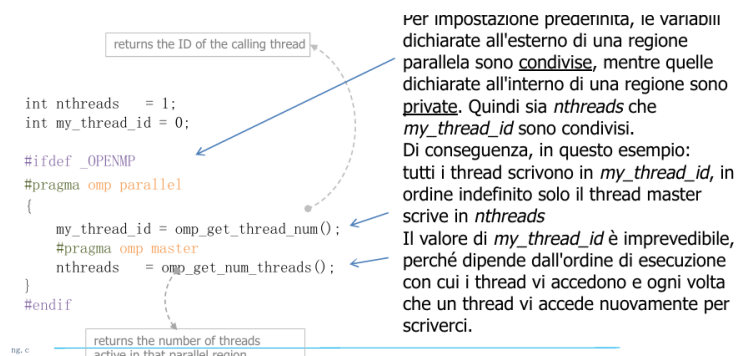
Variabili Condivise o Private

Il cuore di OpenMP è definire correttamente quali variabili risiedono nell'Heap condiviso e quali negli Stack privati.

Possiamo trovare, quindi due tipi di variabili:

- **Shared:** È tutto ciò che è definito nella regione seriale che viene ereditato come **condiviso** all'interno della regione parallela. Tutti i thread possono dunque leggerle e scriverle.
- **Private:** La clausola `private(x)` ordina al compilatore di allocare un *nuovo blocco di memoria non inizializzato* nello stack di ogni thread. Sebbene si chiami `x` come la variabile originale, punta a un indirizzo fisico completamente diverso e all'inizio non è inizializzata, rischiando dunque di generare errori nel caso non vengano inizializzate. In realtà, l'utilizzo di `private`, proprio a causa di questi problemi è **fortemente sconsigliata** e si predilige la dichiarazione all'interno della regione condivisa stessa così da rendere la cosa più chiara ed evitando errori di distrazione.

Un esempio può essere questo:



Visto che le variabili `private` "nascono vuote" (non inizializzate), OpenMP fornisce strumenti per copiare i valori da e verso l'esterno:

- **firstprivate(x):** La variabile è privata (ogni thread ha la sua copia in stack), ma il suo valore di partenza non è casuale: viene **inizializzato al valore che la variabile `x` originale aveva** prima di entrare nella regione parallela.
- **lastprivate(x):** Ha senso solo nei cicli `for` parallelizzati. La variabile originale (nello stack del thread master) prenderà il valore che la sua copia privata ha calcolato durante **l'ultima iterazione logica** del ciclo.

Per loro natura, le variabili private nascono e muoiono all'interno di una regione condivisa. Tuttavia, potrebbe capitare di volere che queste variabili in un qualche modo rimangano e, dunque, esistono dei costrutti anche per fare questo:

- **threadprivate(var)**: Rende una variabile globale privata per ogni thread. A differenza del normale `private`, questa variabile **sopravvive tra diverse regioni parallele** consecutive (il thread non perde il dato). Nota: non si può usare se il numero di thread cambia dinamicamente.
- **copyin(var)**: Usata insieme a `threadprivate`. Serve per far sì che, all'ingresso di una regione parallela, il thread master copi il valore della sua variabile `threadprivate` dentro le copie di tutti gli altri thread.
- **copyprivate(var)**: Si applica ai costrutti eseguiti da un singolo thread (es. direttiva `single`). Propaga il risultato calcolato da quel singolo thread verso le copie private di tutti gli altri thread del team, prima che attraversino la barriera finale.